

ECE 253 – Digital Image Processing

Noise Models, Noise Filtering, Spatial Filtering

Noise Models and Sources of Noise

Additive noise model:

$$g(x, y) = f(x, y) + n(x, y)$$

$g(x, y)$ is the observed noisy image

$f(x, y)$ is the underlying true image

$n(x, y)$ is the noise, assumed to be independent of f

Noise is proportional to signal:

$$\begin{aligned} g(x, y) &= f(x, y) + n(x, y)f(x, y) \\ &= f(x, y)[1 + n(x, y)] \\ &= f(x, y)n'(x, y) \end{aligned}$$

Multiplicative noise model

Gaussian noise:

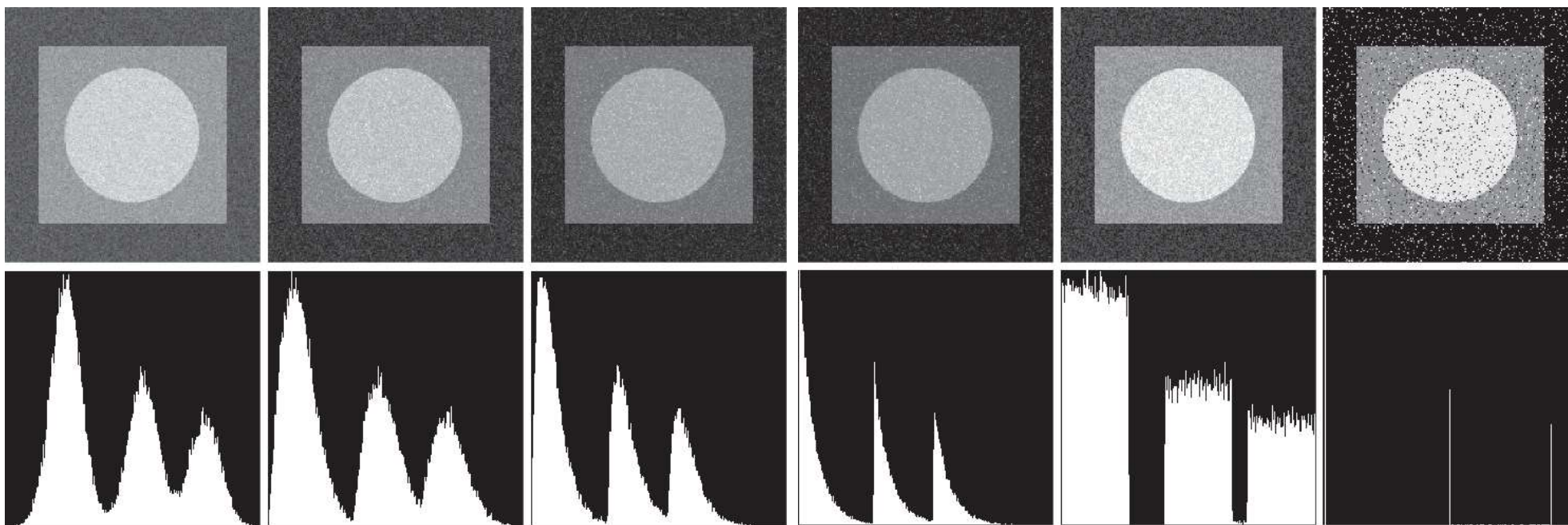
$$p(z) = \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{(z - \bar{z})^2}{2\sigma^2}} \quad \text{for } -\infty < z < \infty$$

Textbook also mentions Rayleigh, Erlang (Gamma), Exponential, and Uniform noise

Examples from Gonzalez & Woods



Test
pattern



Gaussian

Rayleigh

Erlang

Exponential

Uniform

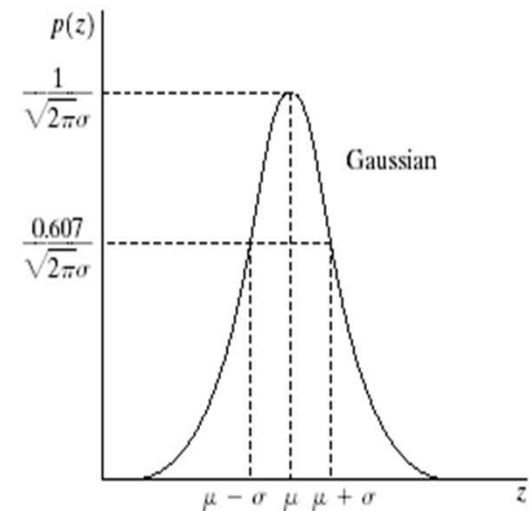
Salt & Pepper

Gaussian Noise

- Noise (image) can be classified according the distribution of the values of pixels (of the noise image) or its (normalized) histogram
- Gaussian noise is characterized by two parameters, μ (mean) and σ^2 (variance), by

$$p(z) = \frac{1}{\sqrt{2\pi}\sigma} e^{-(z-\mu)^2 / 2\sigma^2}$$

- 70% values of z fall in the range $[(\mu-\sigma),(\mu+\sigma)]$
- 95% values of z fall in the range $[(\mu-2\sigma),(\mu+2\sigma)]$



Other Noise Models

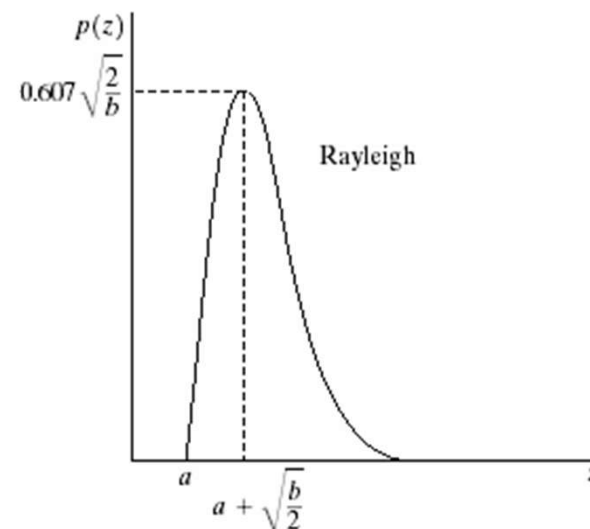
- **Rayleigh** noise

$$p(z) = \begin{cases} \frac{2}{b}(z-a)e^{-(z-a)^2/b} & \text{for } z \geq a \\ 0 & \text{for } z < a \end{cases}$$

- The mean and variance of this density are given by

$$\mu = a + \sqrt{\pi b / 4} \quad \text{and} \quad \sigma^2 = \frac{b(4 - \pi)}{4}$$

- a and b can be obtained through mean and variance



Other Noise Models

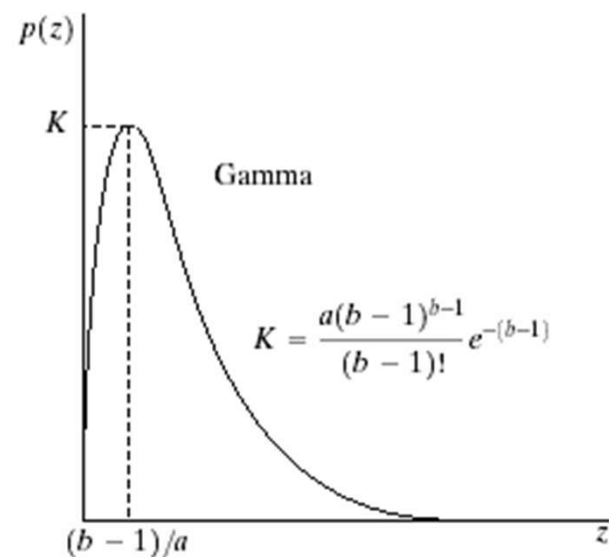
- **Erlang (Gamma) noise**

$$p(z) = \begin{cases} \frac{a^b z^{b-1}}{(b-1)!} e^{-az} & \text{for } z \geq 0 \\ 0 & \text{for } z < 0 \end{cases}$$

- The mean and variance of this density are given by

$$\mu = b/a \text{ and } \sigma^2 = \frac{b}{a^2}$$

- a and b can be obtained through mean and variance



Other Noise Models

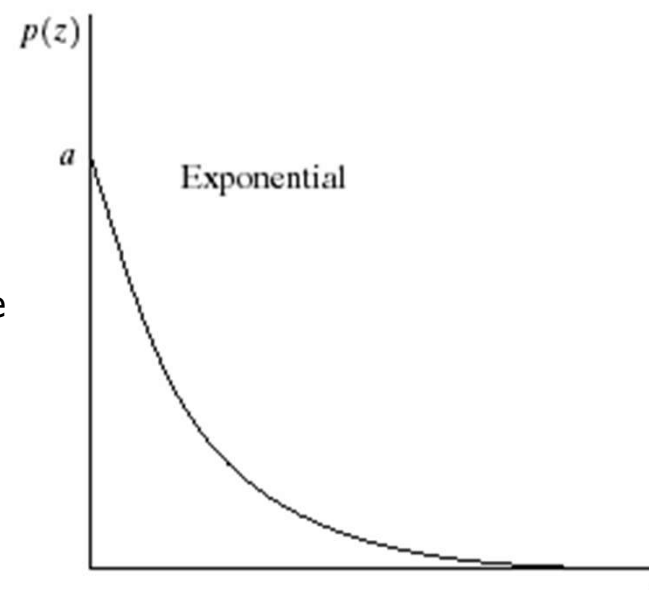
- **Exponential** noise

$$p(z) = \begin{cases} ae^{-az} & \text{for } z \geq 0 \\ 0 & \text{for } z < 0 \end{cases}$$

- The mean and variance of this density are give

$$\mu = 1/a \text{ and } \sigma^2 = \frac{1}{a^2}$$

- Special case pf Erlang PDF with b=1



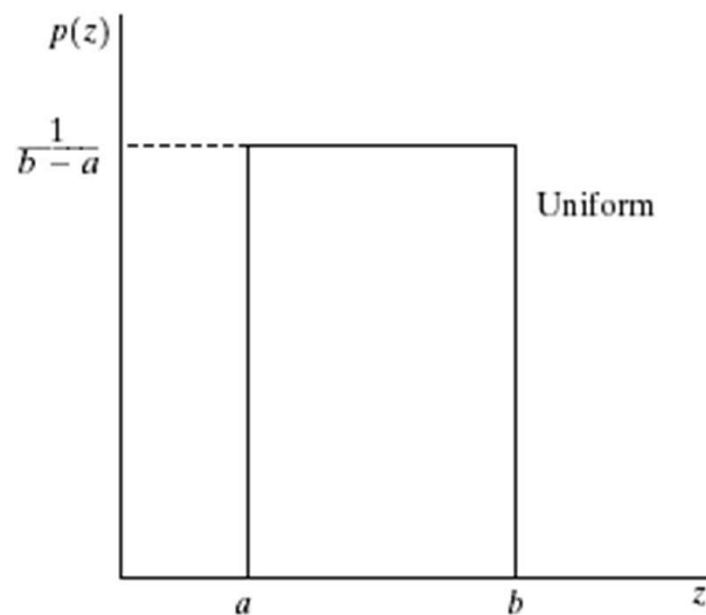
Other Noise Models

- **Uniform** noise

$$p(z) = \begin{cases} \frac{1}{b-a} & \text{if } a \leq z \leq b \\ 0 & \text{otherwise} \end{cases}$$

The mean and variance of this density are given by

$$\mu = (a+b)/2 \text{ and } \sigma^2 = \frac{(b-a)^2}{12}$$



Salt and Pepper Noise, also called Impulse Noise or Outlier Noise

Not well modeled as either additive or multiplicative noise

$g(x, y)$ is the observed noisy image

$f(x, y)$ is the underlying true image

For salt noise:

$$\Pr(g = f) = 1 - P_{salt}$$

$$\Pr(g = MAX) = P_{salt}$$

For pepper noise:

$$\Pr(g = f) = 1 - P_{pepper}$$

$$\Pr(g = MIN) = P_{pepper}$$

For salt-and-pepper noise:

$$\Pr(g = f) = 1 - P_{salt} - P_{pepper}$$

$$\Pr(g = MAX) = P_{salt}$$

$$\Pr(g = MIN) = P_{pepper}$$

Not necessarily MAX/MIN possible values, could just be large/small

Main point: pixels which are noisy are very noisy

Salt-and-pepper Noise



(a) Original image



(b) With added salt & pepper noise

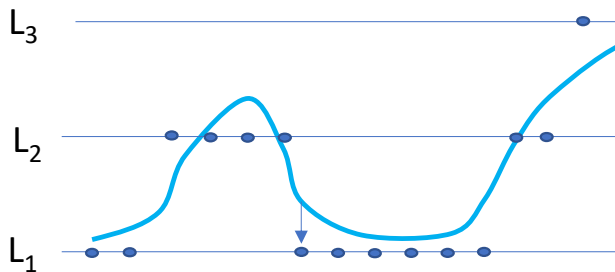
Figure 5.1: Noise on an image

Thermal Noise

- Random thermal noise sources in the circuits that sense, acquire, process the signal from the detector's photoactive surface
- Additive Gaussian noise
- Light intensity is never negative, so if image is an intensity map $\rightarrow$ Noise cannot be strictly Gaussian

Quantization Noise

- Difference between the original signal and the quantized version



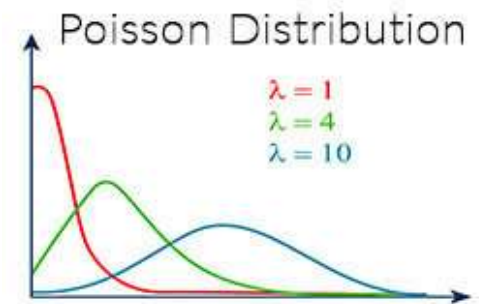
- Deterministic function of the signal
- Usually modeled as additive uniform noise distributed between $-\frac{\Delta}{2}$ and $+\frac{\Delta}{2}$ where Δ is the step size

MUSIC TECHNOLOGY 101:
**UNDERSTANDING
DITHER**

MUSIC TECHNOLOGY 101:
**UNDERSTANDING
DITHER**

Noise from Counting Statistics

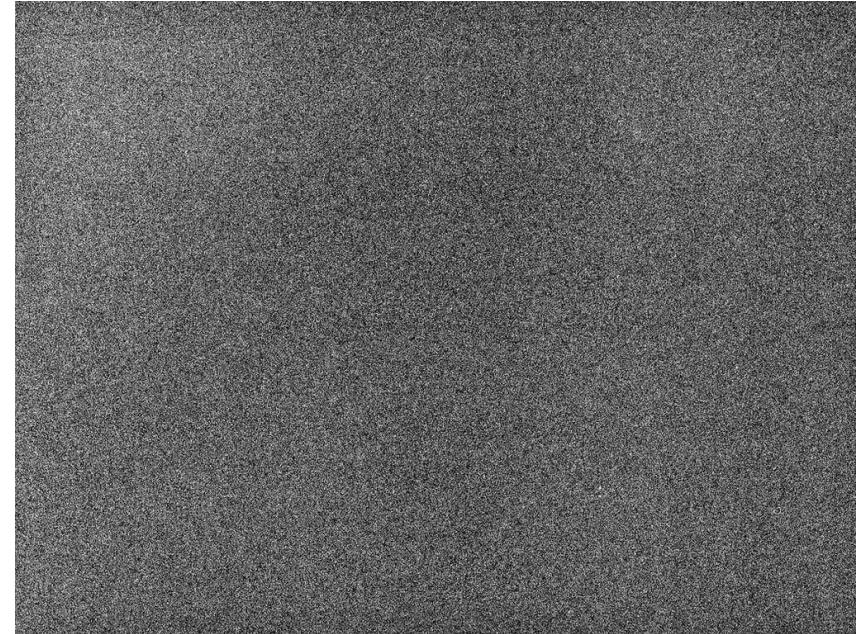
- Uncertainty in the exact number of incident particles (photons, electrons) that arrive at the image detector to form the image
- Statistical fluctuations matter
- Where does this arise?
 - Positron Emission Tomography (PET)
 - Scanning electron microscopy
 - Extremely low light levels
- Photon arrivals modeled as a Poisson process: $\Pr(a = k) = \frac{e^{-\lambda} \lambda^k}{k!}$.
- Property of Poisson distribution: mean = λ , variance = λ
- Brighter region of image has higher $\lambda \rightarrow$ higher variance



Courtesy of Bachelorprint.com

Film Grain Noise

- Silver halide grains are distributed over the surface of the film
- Exposure is binary process:
 - Grains exposed to enough light are converted to grains of metallic silver
 - Unexposed grains are washed off during development
- Randomness in:
 - Distribution of the grains
 - Varying size / shape of the grains
 - Number of photons required to expose a particular grain
- Signal-dependent noise, often modeled as multiplicative process

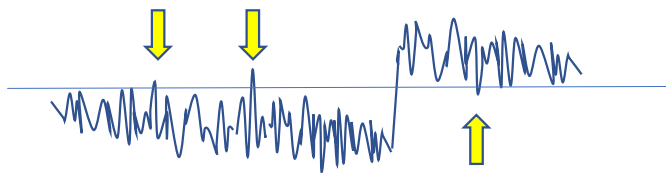






Salt and Pepper Noise

1. Dust or dirt on the optics
2. Electronic cameras: malfunctioning pixels (e.g., faulty memory locations)
3. Grayscale image gets thresholded:



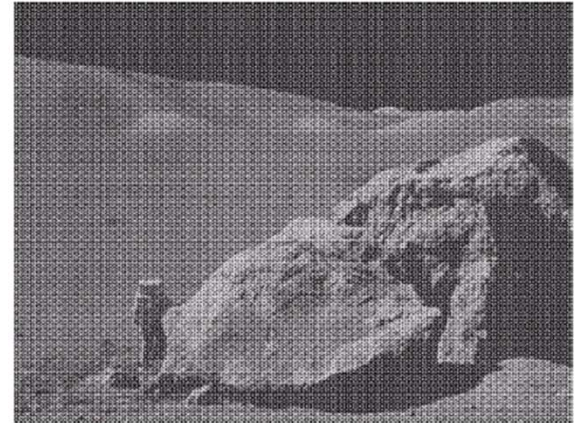
Although in the grayscale image this noise would not be called salt-and-pepper, in the binary version it shows up as isolated pixels of the wrong color

4. Channel errors: Consider a binary symmetric channel (BSC) that flips individual bits
 $10110100 \rightarrow 00110100$ Most Significant Bit (MSB) got flipped, change is -128

Periodic Noise

a
b

- Arises typically from electrical or electromechanical interference during image acquisition
- It can be observed by visual inspection both in the spatial domain and frequency domain
- The only spatially dependent noise will be considered



Noise Removal

Salt and pepper noise, Gaussian noise and speckle noise can all be cleaned by using **spatial filtering** techniques.

Periodic noise, however, requires the use of **frequency domain filtering** because whereas the other forms of noise can be modelled as local degradations, periodic noise is a global effect.

Noise Removal



(a) 3×3 averaging



(b) 7×7 averaging

Figure 5.4: Attempting to clean salt & pepper noise with average filtering

Noise Removal



Figure 5.5: Cleaning salt and pepper noise with a median filter

Noise Removal



(a) 20% salt & pepper noise



(b) After median filtering

Figure 5.6: Using a 3×3 median filter on more noise

Noise Removal



(a) Using `medfilt2` twice



(b) using a 5×5 median filter

Figure 5.7: Cleaning 20% salt & pepper noise with median filtering

Image filtering and enhancement

Spatial filtering



Example: low-pass filter

1D correlation and convolution

1D correlation (also called cross-correlation)

$$g(x) = \sum_{s=-a}^a w(s)f(x+s)$$

$$g(x) = w(x) \star f(x), \text{ where } w(x) \star f(x) = \sum_{s=-a}^a w(s)f(x+s)$$

1D convolution

$$g(x) = \sum_{s=-a}^a w(s)f(x-s)$$

$$g(x) = w(x) \star f(x), \text{ where } w(x) \star f(x) = \sum_{s=-a}^a w(s)f(x-s)$$

Equivalent to correlation with w rotated 180 degrees
where

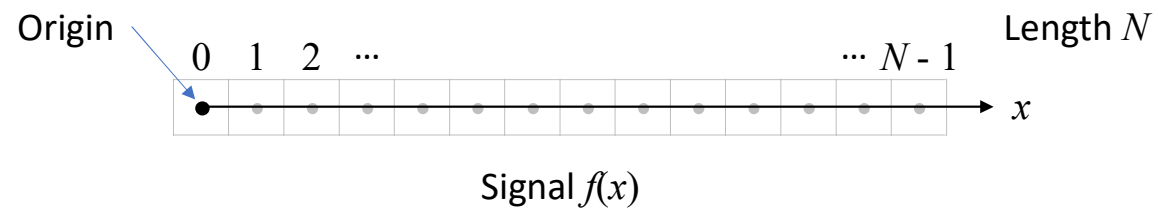
When kernel is symmetric, convolution
and correlation give the same result

$w(s)$ is spatial filtering kernel (also called mask, template, and window)

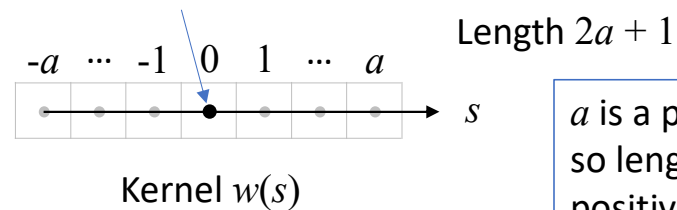
$f(x)$ is input function (signal)

$g(x)$ is output function (signal)

1D spatial filtering



Origin (also called anchor) at center



a is a positive integer,
so length is an odd
positive integer

1D correlation $w(x) \star f(x) = \sum_{s=-a}^a w(s) f(x + s)$

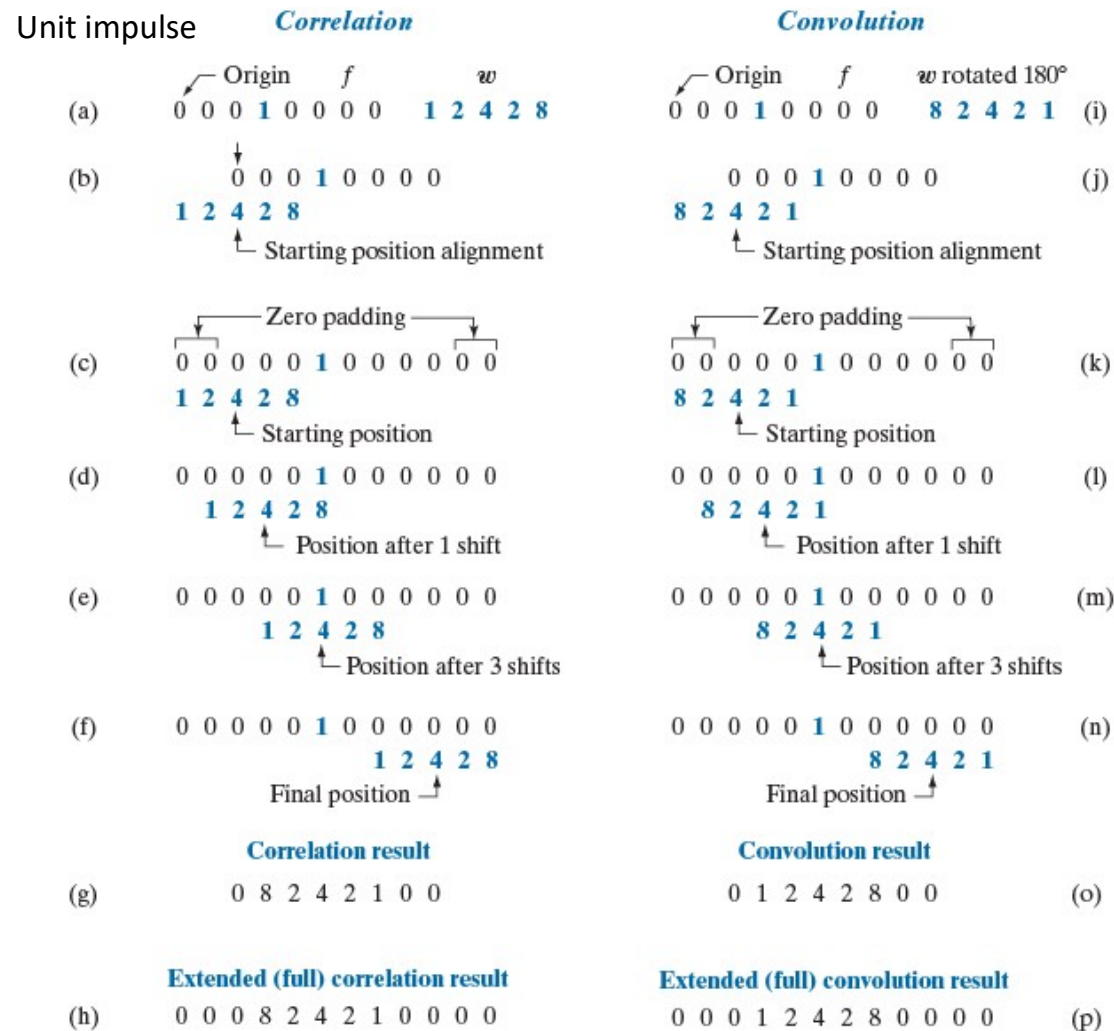
1D convolution $w(x) \star f(x) = \sum_{s=-a}^a w(s) f(x - s)$

For convolution, kernel
is rotated 180 degrees

1D correlation and convolution

FIGURE 3.35

Illustration of 1-D correlation and convolution of a kernel, w , with a function f consisting of a discrete unit impulse. Note that correlation and convolution are functions of the variable x , which acts to *displace* one function with respect to the other. For the extended correlation and convolution results, the starting configuration places the right-most element of the kernel to be coincident with the origin of f . Additional padding must be used.



For convolution, kernel is rotated 180 degrees

2D correlation and convolution

2D correlation (also called cross-correlation)

$$g(x, y) = \sum_{s=-a}^a \sum_{t=-b}^b w(s, t) f(x + s, y + t)$$

$$g(x, y) = w(x, y) \star f(x, y), \text{ where } w(x, y) \star f(x, y) = \sum_{s=-a}^a \sum_{t=-b}^b w(s, t) f(x + s, y + t)$$

2D convolution

$$g(x, y) = \sum_{s=-a}^a \sum_{t=-b}^b w(s, t) f(x - s, y - t)$$

$$g(x, y) = w(x, y) \star f(x, y), \text{ where } w(x, y) \star f(x, y) = \sum_{s=-a}^a \sum_{t=-b}^b w(s, t) f(x - s, y - t)$$

Equivalent to correlation with w rotated 180 degrees
where

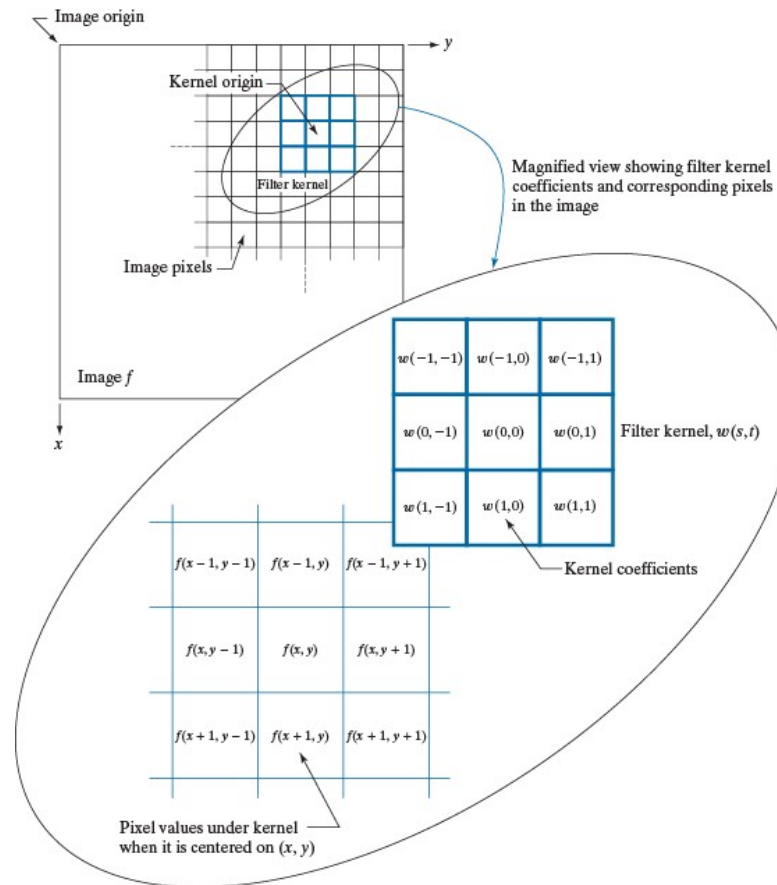
When kernel is symmetric, convolution
and correlation give the same result

$w(s, t)$ is spatial filtering kernel (also called mask, template, and window)

$f(x, y)$ is input image

$g(x, y)$ is output image

2D spatial filtering



$m \times n$ kernel is $(2a + 1) \times (2b + 1)$, where a and b are positive integers, so m and n are odd positive integers

2D correlation

$$w(x, y) \star f(x, y) = \sum_{s=-a}^a \sum_{t=-b}^b w(s, t) f(x + s, y + t)$$

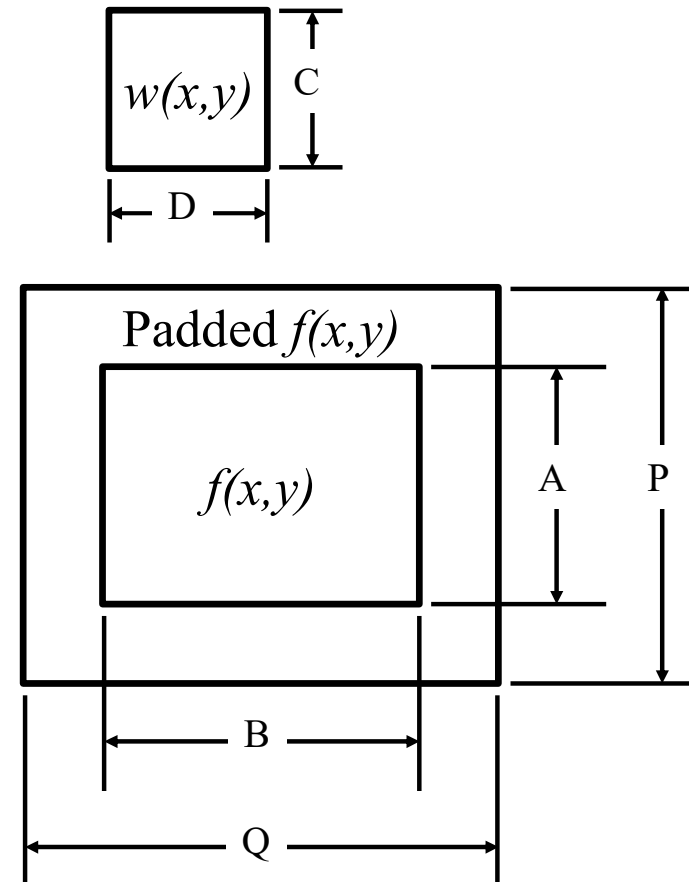
2D convolution

$$w(x, y) \star f(x, y) = \sum_{s=-a}^a \sum_{t=-b}^b w(s, t) f(x - s, y - t)$$

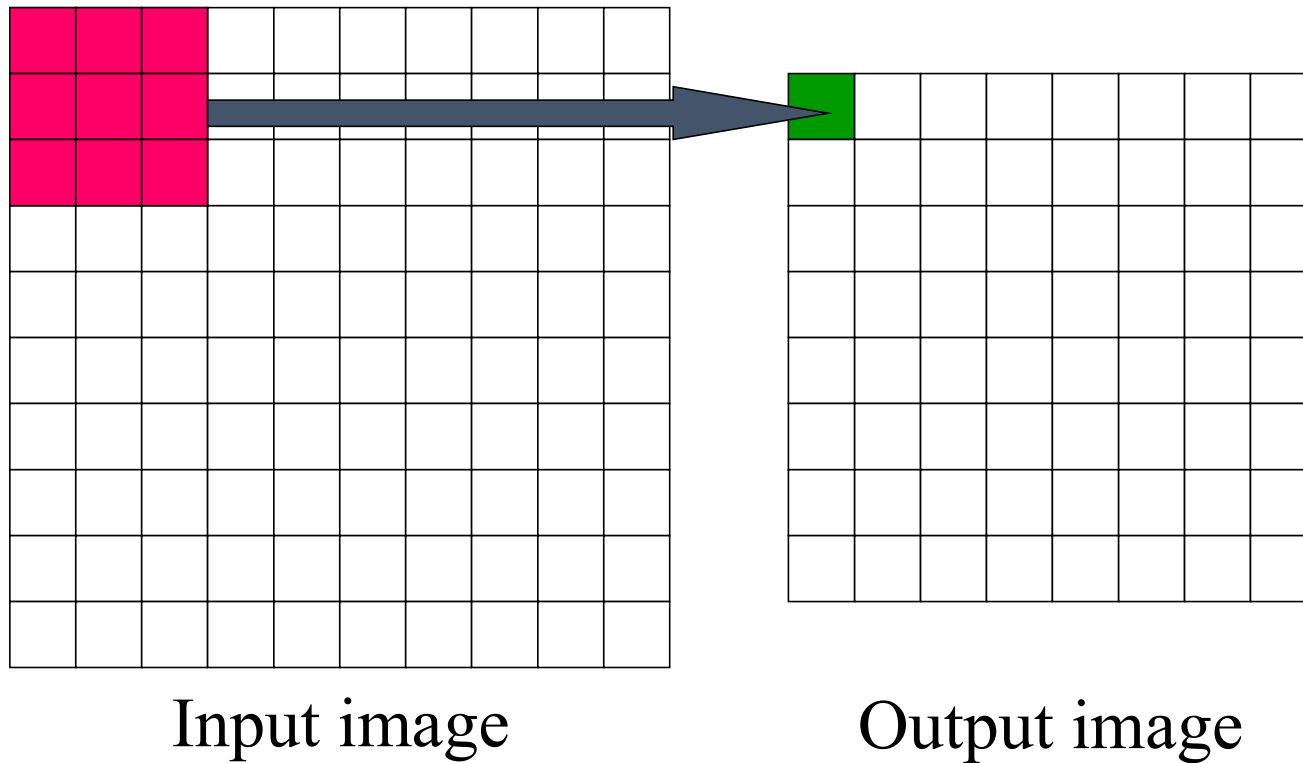
For convolution, kernel is rotated 180 degrees

2D spatial filtering, shape

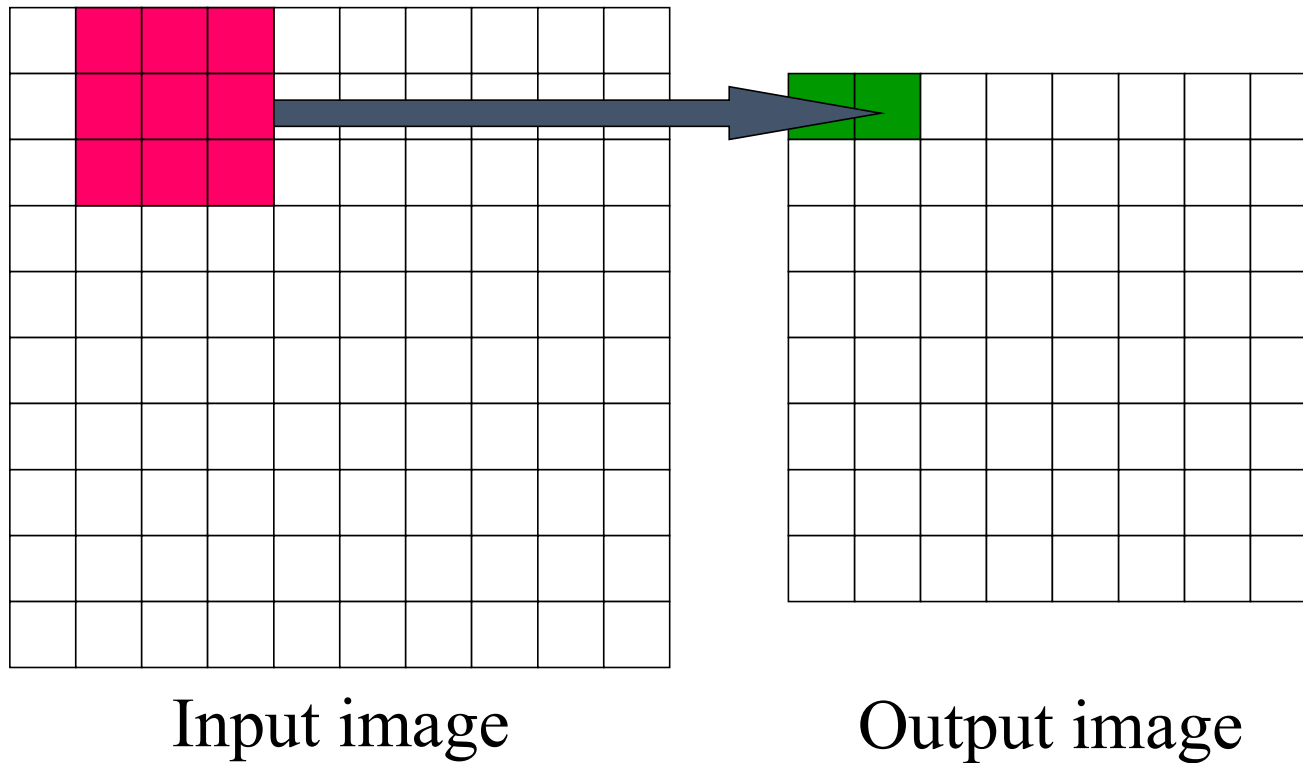
- Full
 - $w(x,y)$ and $f(x,y)$ have at least 1 pixel overlap
 - $P = A + 2(C - 1)$
 - $Q = B + 2(D - 1)$
 - Output $g(x,y)$
 - Width is $B + D - 1$, height is $A + C - 1$
- Same
 - $P = A + C - 1$
 - $Q = B + D - 1$
 - Output $g(x,y)$
 - Width is B , height is A
- Valid
 - $w(x,y)$ must be completely inside $f(x,y)$
 - $P = A$
 - $Q = B$
 - Output $g(x,y)$
 - Width is $B - D + 1$, height is $A - C + 1$



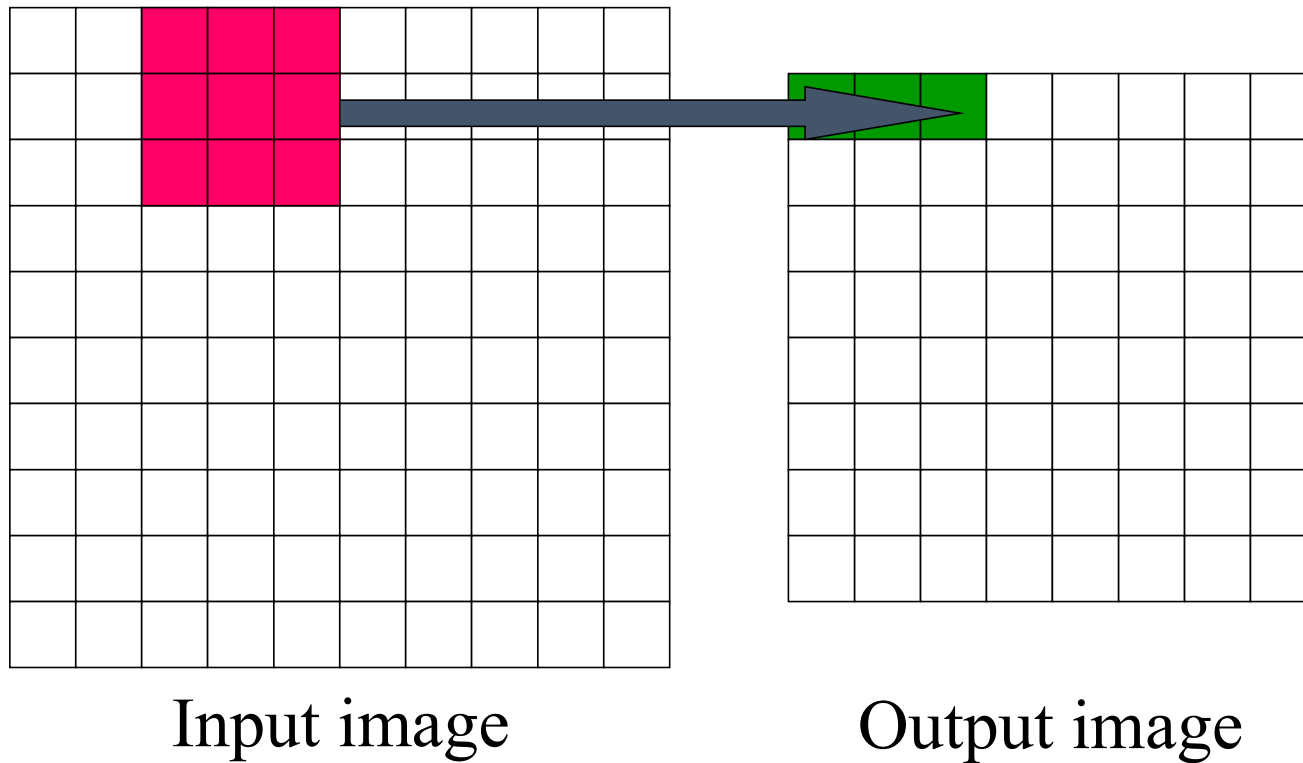
2D spatial filtering, valid shape



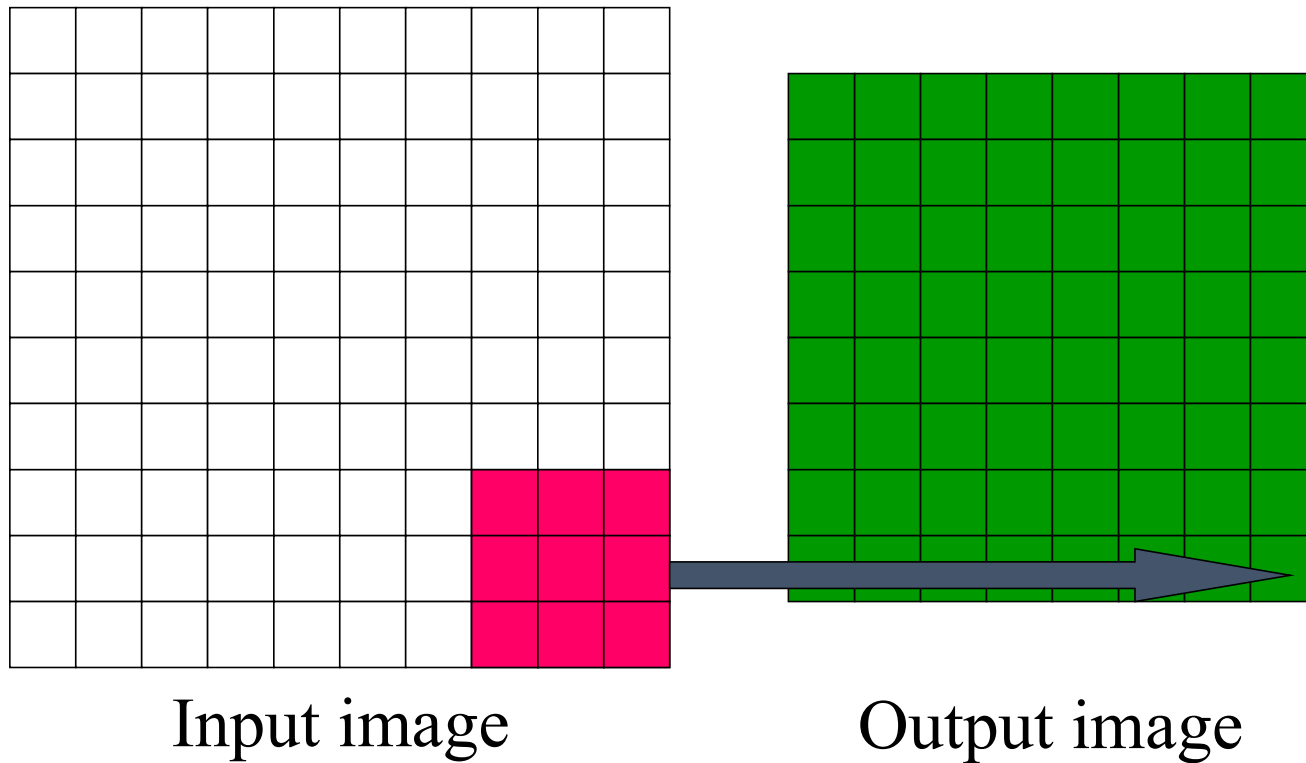
2D spatial filtering, valid shape



2D spatial filtering, valid shape



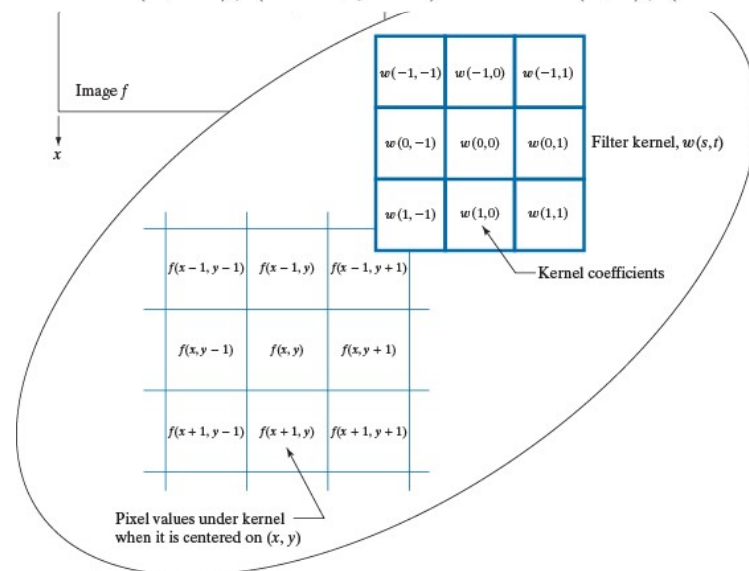
2D spatial filtering, valid shape



2D spatial filtering

- Example: 3 x 3 correlation

$$\begin{aligned}
 g(x, y) = & w(-1, -1)f(x - 1, y - 1) + w(-1, 0)f(x - 1, y) + w(-1, 1)f(x - 1, y + 1) \\
 & + w(0, -1)f(x, y - 1) + w(0, 0)f(x, y) + w(0, 1)f(x, y + 1) \\
 & + w(1, -1)f(x + 1, y - 1) + w(1, 0)f(x + 1, y) + w(1, 1)f(x + 1, y + 1)
 \end{aligned}$$



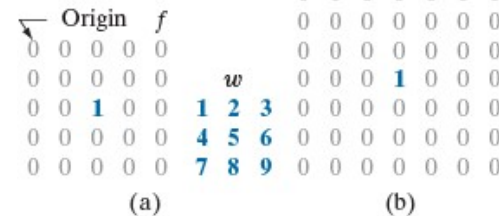
Largest response when kernel coefficients are equal to corresponding image pixel values

2D correlation and convolution

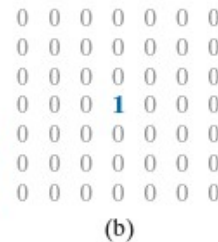
FIGURE 3.36

Correlation (middle row) and convolution (last row) of a 2-D kernel with an image consisting of a discrete unit impulse. The 0's are shown in gray to simplify visual analysis. Note that correlation and convolution are functions of x and y . As these variable change, they *displace* one function with respect to the other. See the discussion of Eqs. (3-45) and (3-46) regarding full correlation and convolution.

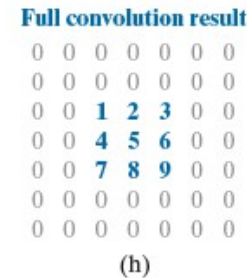
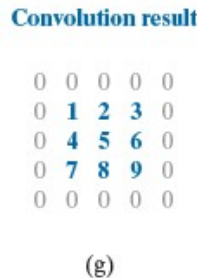
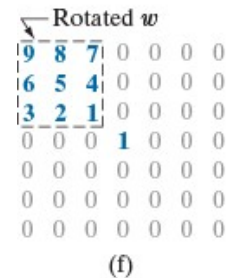
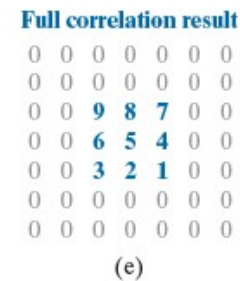
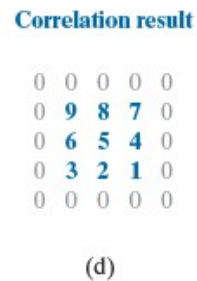
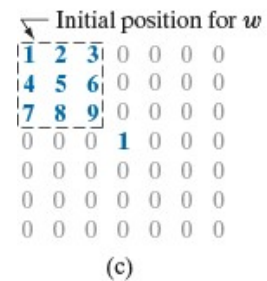
Unit impulse



Padded f



For convolution, kernel is rotated 180 degrees



Correlation and convolution

This is equivalent to applying one filter

TABLE 3.5

Some fundamental properties of convolution and correlation. A dash means that the property does not hold.

Property	Convolution	Correlation
Commutative	$f \star g = g \star f$	—
Associative	$f \star (g \star h) = (f \star g) \star h$	—
Distributive	$f \star (g + h) = (f \star g) + (f \star h)$	$f \star (g + h) = (f \star g) + (f \star h)$

- Convolution is commutative and associative, correlation is not

Noise Removal & Spatial Filtering

- Averaging
- Selective averaging
 - Directional smoothing
 - K-nearest neighbor
- Median filtering

Averaging

- Suppose the non-noisy gray levels are $z_1, z_2, \dots, z_n$
- Additive noise values are (i.i.d.) $w_1, w_2, \dots, w_n \sim N(0, \sigma^2)$
- Observed values are $z_1 + w_1, z_2 + w_2, \dots, z_n + w_n$
- Averaging them produces:

$$\frac{z_1 + z_2 + \dots + z_n}{n}$$

Image Blur

+

$$\frac{w_1 + w_2 + \dots + w_n}{n}$$

Sample of a random variable with
mean zero and S.D. $\sigma/\sqrt{n}$

Averaging over uniform area

$$\begin{bmatrix} 100 & 100 & 100 & 100 & 100 \\ 100 & 100 & 100 & 100 & 100 \\ 100 & 100 & 100 & 100 & 100 \\ 100 & 100 & 100 & 100 & 100 \\ 100 & 100 & 100 & 100 & 100 \end{bmatrix} + \begin{bmatrix} +4 & -6 & +7 & +1 & -4 \\ -4 & -2 & +6 & -10 & -6 \\ -4 & +9 & +6 & -1 & -6 \\ -4 & -4 & -1 & 15 & +1 \\ +6 & +2 & -7 & +7 & 0 \end{bmatrix} = \begin{bmatrix} 104 & 94 & 107 & 101 & 96 \\ 96 & 98 & 106 & 90 & 94 \\ 96 & 109 & 106 & 99 & 94 \\ 96 & 96 & 99 & 115 & 101 \\ 106 & 102 & 93 & 107 & 100 \end{bmatrix}$$

$$\frac{98 + 106 + 90 + 109 + 106 + 99 + 96 + 99 + 115}{9} = 102$$

Box Filter

$$\frac{1}{9} \begin{pmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{pmatrix}$$

- Averaging over a 3x3 window reduces the noise variance by a factor of 9

Spatial smoothing filters = Lowpass filters

$$\frac{1}{9} \begin{pmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{pmatrix}$$

- Sum elements in array, multiply by coefficient out front, equals 1
- average value is unchanged
- range [0,255] can't get any larger

Low spatial frequencies pass through

100	100	100	100	100
100	100	100	100	100
100	100	100	100	100
100	100	100	100	100
100	100	100	100	100
→				
100	100	100	100	100
100	100	100	100	100
100	100	100	100	100
100	100	100	100	100
100	100	100	100	100
→				
80	90	100	110	120
80	90	100	110	120
80	90	100	110	120
80	90	100	110	120
80	90	100	110	120
80	90	100	110	120
80	90	100	110	120
80	90	100	110	120

High spatial frequencies get attenuated:

0	200	0	200	0
200	0	200	0	200
0	200	0	200	0
200	0	200	0	200
0	200	0	200	0
↓				
89	111	89	111	89
111	89	111	89	111
89	111	89	111	89
111	89	111	89	111
89	111	89	111	89

Many different flavors: How do we choose?

Separable Filter Kernels

A 2D function $G(x,y)$ is said to be separable if it can be written as the product of two 1D functions:

$$G(x,y) = G_1(x)G_2(y)$$

A separable kernel is a matrix that can be written as the outer product of two vectors:

$$\begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix} \begin{bmatrix} 1 & 2 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

$$\begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix} \begin{bmatrix} 1 & 7 \end{bmatrix} = \begin{bmatrix} 1 & 7 \\ 2 & 14 \\ 1 & 7 \end{bmatrix}$$

Convolving a separable kernel with an image is the same as convolving with one of the 1D kernels first, and then convolving the result with the other 1D kernel

Image size $M \times N$, Kernel size $m \times n$

Filtering with 2D kernel requires on the order of **$MNmn$** multiplications and additions

Filtering with 1D kernels sequentially requires **$MNm + MNn = MN(m+n)$** operations

- Box filters
- Gaussian filters (circularly symmetric)

Spatial smoothing filters = Lowpass filters

$$\frac{1}{9} \begin{pmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{pmatrix}$$

$$\frac{1}{8} \begin{pmatrix} 0 & 1 & 0 \\ 1 & 4 & 1 \\ 0 & 1 & 0 \end{pmatrix}$$

$$\frac{1}{10} \begin{pmatrix} 1 & 1 & 1 \\ 1 & 2 & 1 \\ 1 & 1 & 1 \end{pmatrix}$$

$$\frac{1}{16} \begin{pmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{pmatrix}$$

- Sum elements in array, multiply by coefficient out front, equals 1
 - average value is unchanged
 - range [0,255] can't get any larger
- Effect of central weighting
 - less blurring
 - less noise smoothing
- Can have larger sizes (more blurring, more noise smoothing)

$$\frac{1}{(b+2)^2} \begin{pmatrix} 1 & b & 1 \\ b & b^2 & b \\ 1 & b & 1 \end{pmatrix}$$

As $b \rightarrow \infty$ this becomes an identity filter (allpass)

Larger filters

Consider the filter $(1/25) * \text{ones}(5)$

100	100	100	100	100	→	100	100	100	100	100
100	100	100	100	100		100	100	100	100	100
100	100	100	100	100		100	100	100	100	100
100	100	100	100	100		100	100	100	100	100
100	100	100	100	100		100	100	100	100	100

80	90	100	110	120	→	80	90	100	110	120
80	90	100	110	120		80	90	100	110	120
80	90	100	110	120		80	90	100	110	120
80	90	100	110	120		80	90	100	110	120
80	90	100	110	120		80	90	100	110	120

High spatial frequencies get attenuated even more:

0	200	0	200	0
200	0	200	0	200
0	200	0	200	0
200	0	200	0	200
0	200	0	200	0

96	104	96	104	96
104	96	104	96	104
96	104	96	104	96
104	96	104	96	104
96	104	96	104	96

Example: Object Boundary

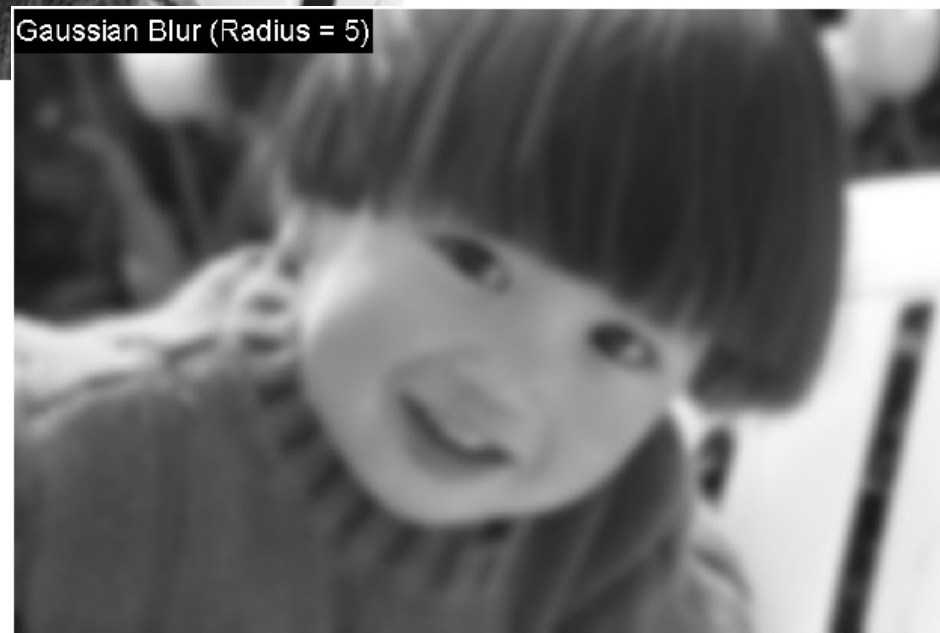
$$\begin{bmatrix} 200 & 200 & 100 & 100 & 100 \\ 200 & 200 & 100 & 100 & 100 \\ 200 & 200 & 100 & 100 & 100 \\ 200 & 200 & 100 & 100 & 100 \\ 200 & 200 & 100 & 100 & 100 \end{bmatrix} + \begin{bmatrix} +4 & -6 & +7 & +1 & -4 \\ -4 & -2 & +6 & -10 & -6 \\ -4 & +9 & +6 & -1 & -6 \\ -4 & -4 & -1 & 15 & +1 \\ +6 & +2 & -7 & +7 & 0 \end{bmatrix} = \begin{bmatrix} 204 & 194 & 107 & 101 & 96 \\ 196 & 198 & 106 & 90 & 94 \\ 196 & 209 & 106 & 99 & 94 \\ 196 & 196 & 99 & 115 & 101 \\ 206 & 202 & 93 & 107 & 100 \end{bmatrix}$$

$$\frac{198 + 106 + 90 + 209 + 106 + 99 + 196 + 99 + 115}{9} = 135$$

- Edge has been blurred

If we could average on only one side of the edge (over 6 pixels),
filtered pixel value would be 102.5

Blur Examples



Many different flavors: How do we choose?

- Try different things, see what works!
- Measure the blur, measure the noise reduction

Input image
has step edge
(infinite
vertical)

0	0	0	0	0	90	90	90	90	90
0	0	0	0	0	90	90	90	90	90
0	0	0	0	0	90	90	90	90	90

Box filters:

3 X 3 →

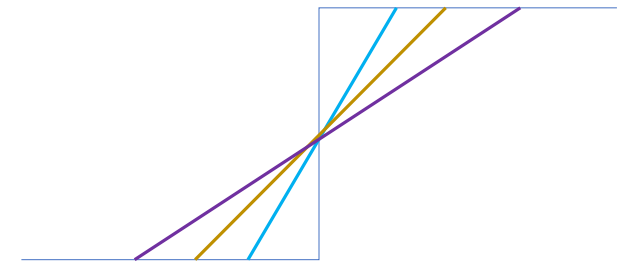
0	0	0	0	30	60	90	90	90	90
---	---	---	---	----	----	----	----	----	----

5 X 5 →

0	0	0	18	36	54	72	90	90	90
---	---	---	----	----	----	----	----	----	----

7 X 7 →

0	0	13	26	39	51	64	77	90	90
---	---	----	----	----	----	----	----	----	----



- Acutance = average steepness of slope

Smoothing kernels

$$\frac{1}{9} \times$$

1	1	1
1	1	1
1	1	1

Average
(box kernel)

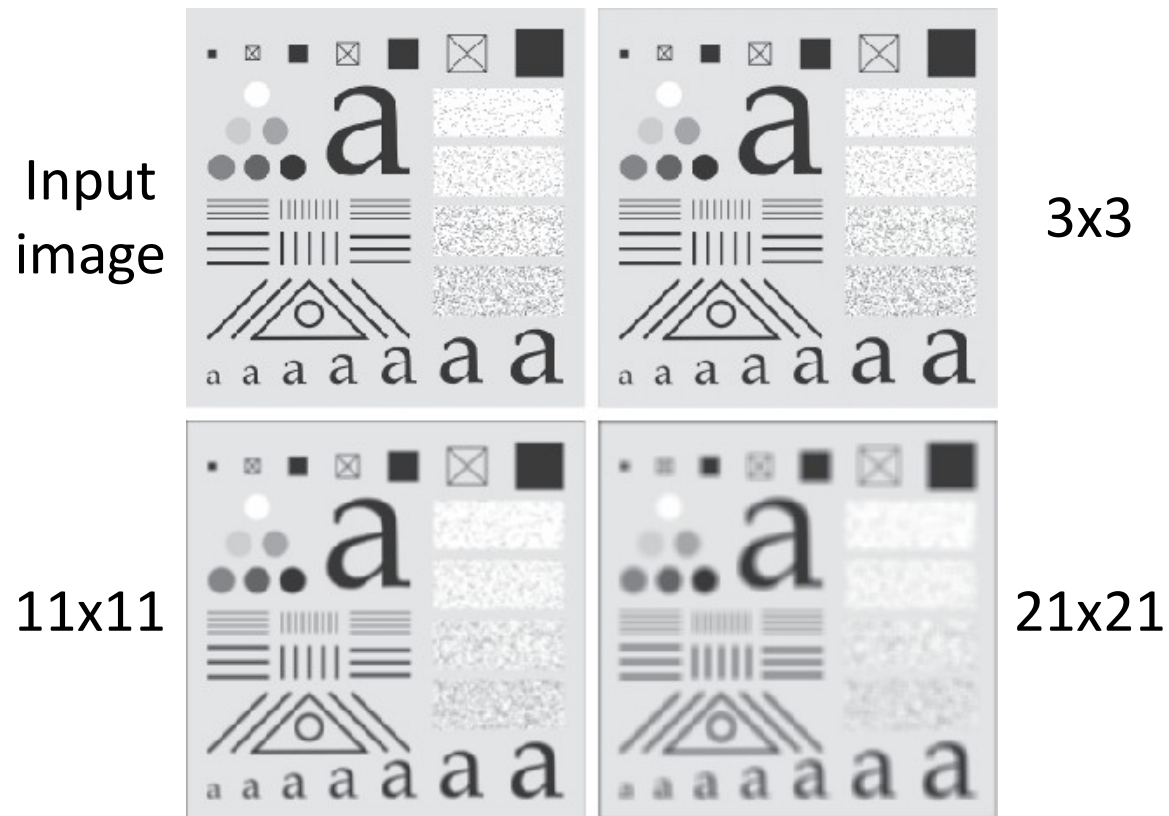
$$\frac{1}{4.8976} \times$$

0.3679	0.6065	0.3679
0.6065	1.0000	0.6065
0.3679	0.6065	0.3679

Weighted average
(Gaussian kernel)

Common that sum of filter coefficients is 1, which prevents (full) bias
Bias: sum of input pixel values not equal to sum of output pixel values

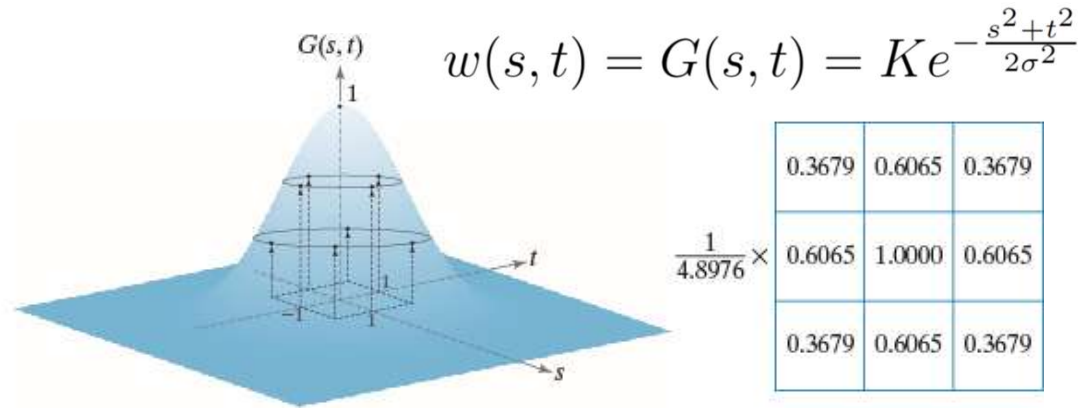
Smoothing with box kernel



Smoothing with Gaussian kernel

a b

FIGURE 3.41
(a) Sampling a Gaussian function to obtain a discrete Gaussian kernel. The values shown are for $K = 1$ and $\sigma = 1$. (b) Resulting 3×3 kernel [this is the same as Fig. 3.37(b)].



Standard deviation σ	Percent of total volume under surface
1	39.35
2	86.47
3	98.89

Volume under surface greater than 3σ is negligible

Smoothing with Gaussian kernel

minimum size is
$$\begin{cases} \lceil 6\sigma \rceil & \text{if } \lceil 6\sigma \rceil \text{ is odd} \\ \lceil 6\sigma \rceil + 1 & \text{otherwise} \end{cases}$$



$\sigma = 7$
43x43

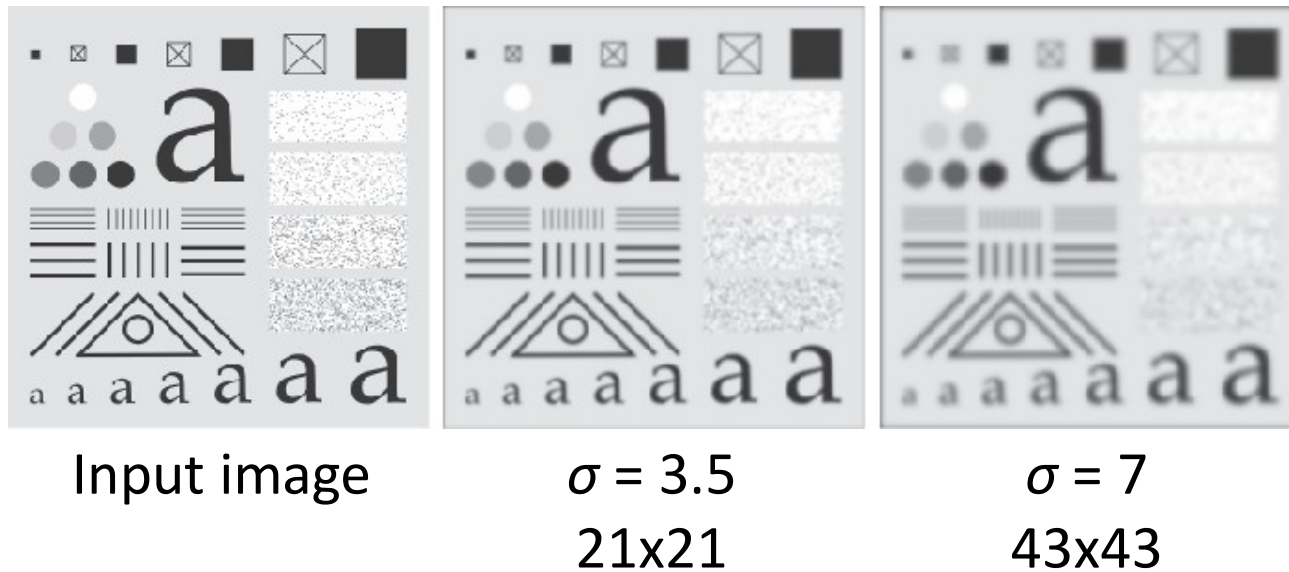


$\sigma = 7$
85x85



Difference

Smoothing with Gaussian kernel



Smoothing with box kernel

Input image



Box kernel

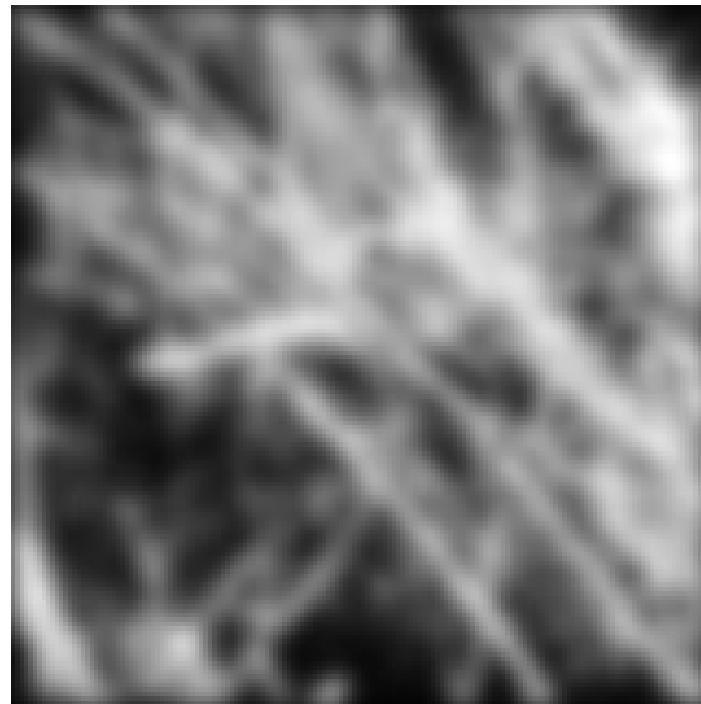


Smoothing with Gaussian kernel

Input image



Gaussian kernel

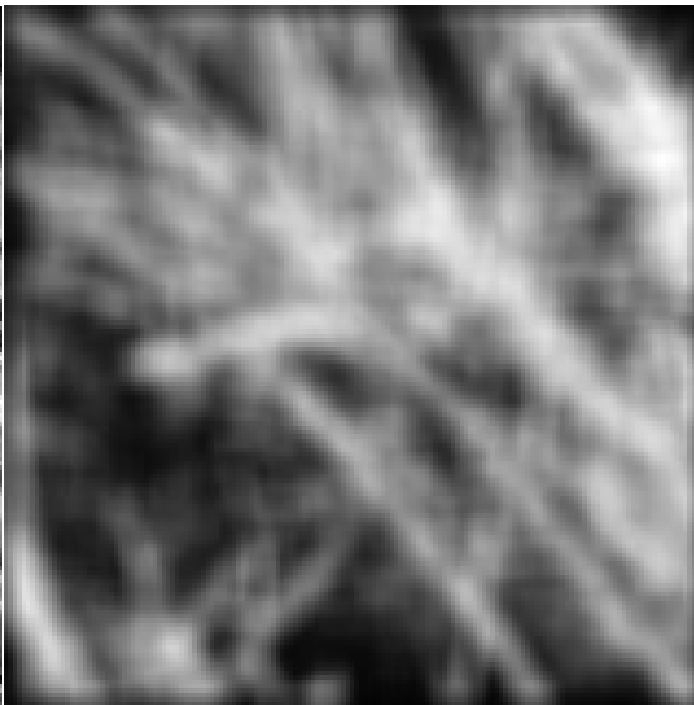


Smoothing

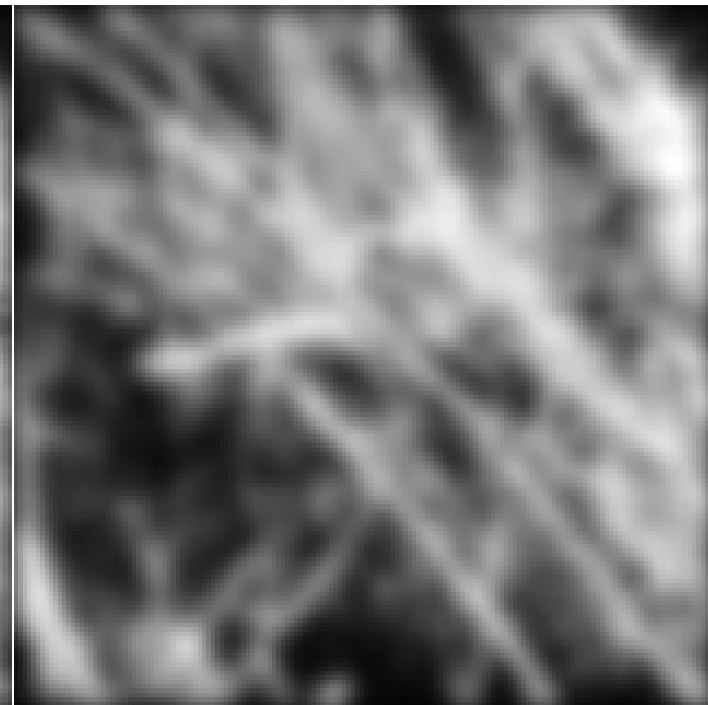
Input Image



Box kernel



Gaussian kernel



Blur – examining resolution

Input image has repeating vertical bars (infinite vertical):

0	0	0	90	90	90	0	0	0	90	90	90
0	0	0	90	90	90	0	0	0	90	90	90
0	0	0	90	90	90	0	0	0	90	90	90

3 X 3 →

30	0	30	60	90	60	30	0	30	60	90	60
----	---	----	----	----	----	----	---	----	----	----	----

Why is this 30?

5 X 5 →

36	36	36	54	54	54	36	36	36	54	54	54
----	----	----	----	----	----	----	----	----	----	----	----

7 X 7 →

51	51	51	39	39	39	51	51	51	39	39	39
----	----	----	----	----	----	----	----	----	----	----	----

- Bright bars are in the wrong place!!!
- Called *spurious resolution*

Blur – examining resolution

Input image has repeating vertical bars (infinite vertical):

0	0	0	90	90	90	0	0	0	90	90	90
0	0	0	90	90	90	0	0	0	90	90	90
0	0	0	90	90	90	0	0	0	90	90	90

3 X 3 →

30	0	30	60	90	60	30	0	30	60	90	60
----	---	----	----	----	----	----	---	----	----	----	----

Answer: Assuming that the bars are repeating for a while:

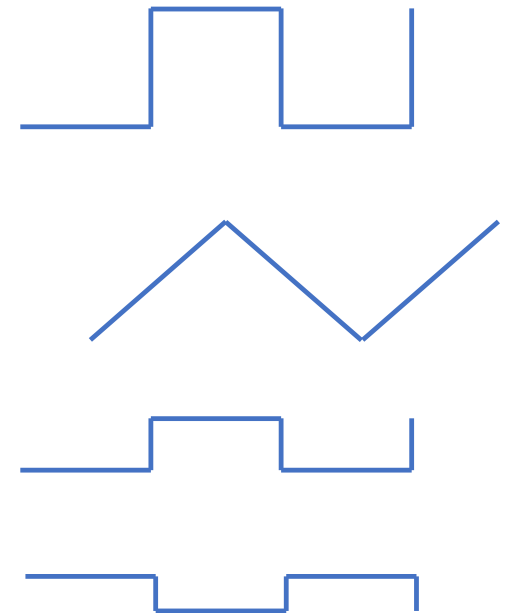
90	90	90	0	0	0	90	90	90	0	0	0	90	90	90	0	0	0
90	90	90	0	0	0	90	90	90	0	0	0	90	90	90	0	0	0
90	90	90	0	0	0	90	90	90	0	0	0	90	90	90	0	0	0

Blur – examining resolution

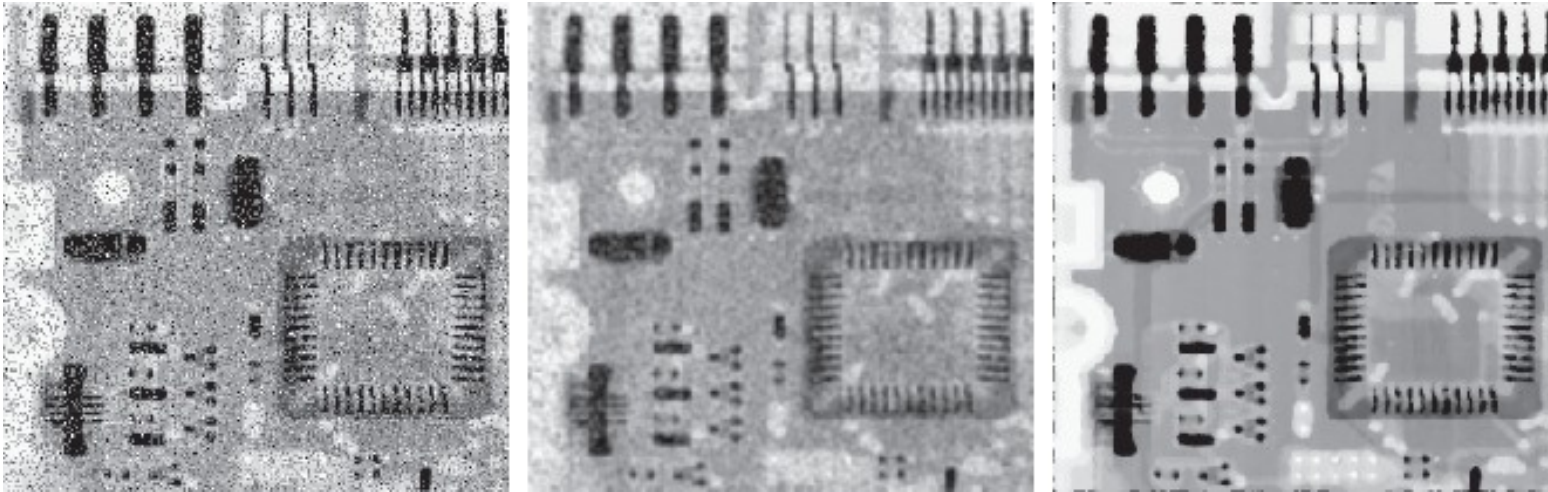
Input image has repeating vertical bars (infinite vertical):

	0	0	0	90	90	90	0	0	0	90	90	90
	0	0	0	90	90	90	0	0	0	90	90	90
	0	0	0	90	90	90	0	0	0	90	90	90
3 X 3 →	30	0	30	60	90	60	30	0	30	60	90	60
5 X 5 →	36	36	36	54	54	54	36	36	36	54	54	54
7 X 7 →	51	51	51	39	39	39	51	51	51	39	39	39

- Bright bars are in the wrong place!!!
- Called *spurious resolution*



Lots of applications with repeating bars



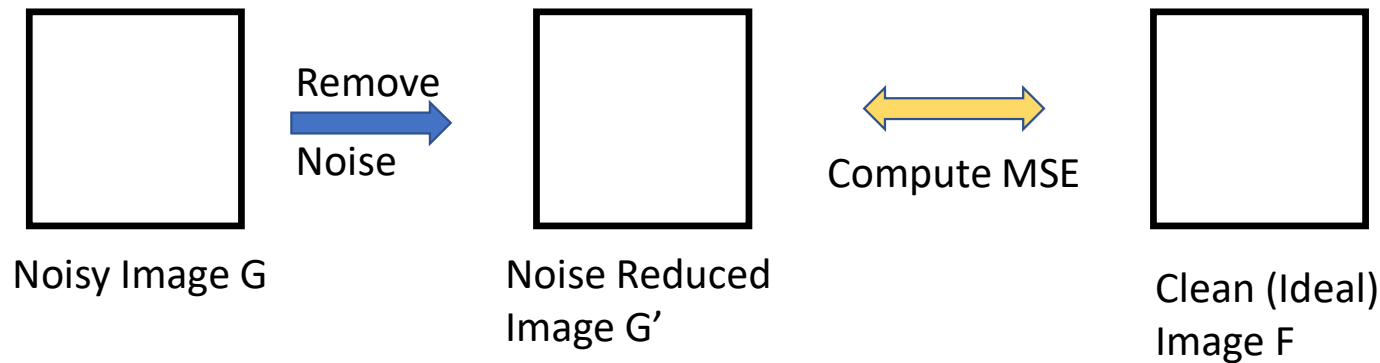
From Gonzalez and Woods, Fig 3.49

a b c

FIGURE 3.49 (a) X-ray image of a circuit board, corrupted by salt-and-pepper noise. (b) Noise reduction using a 19×19 Gaussian lowpass filter kernel with $\sigma = 3$. (c) Noise reduction using a 7×7 median filter. (Original image courtesy of Mr. Joseph E. Pascente, Lixi, Inc.)

Measuring noise reduction

- Can compute mean-squared error (MSE) before and after filtering:



$$MSE = \frac{1}{N \times M} \sum_{i=1}^N \sum_{j=1}^M (F(i,j) - G'(i,j))^2$$

Example: Object Boundary

$$\begin{bmatrix} 200 & 200 & 100 & 100 & 100 \\ 200 & 200 & 100 & 100 & 100 \\ 200 & 200 & 100 & 100 & 100 \\ 200 & 200 & 100 & 100 & 100 \\ 200 & 200 & 100 & 100 & 100 \end{bmatrix} + \begin{bmatrix} +4 & -6 & +7 & +1 & -4 \\ -4 & -2 & +6 & -10 & -6 \\ -4 & +9 & +6 & -1 & -6 \\ -4 & -4 & -1 & 15 & +1 \\ +6 & +2 & -7 & +7 & 0 \end{bmatrix} = \begin{bmatrix} 204 & 194 & 107 & 101 & 96 \\ 196 & 198 & 106 & 90 & 94 \\ 196 & 209 & 106 & 99 & 94 \\ 196 & 196 & 99 & 115 & 101 \\ 206 & 202 & 93 & 107 & 100 \end{bmatrix}$$

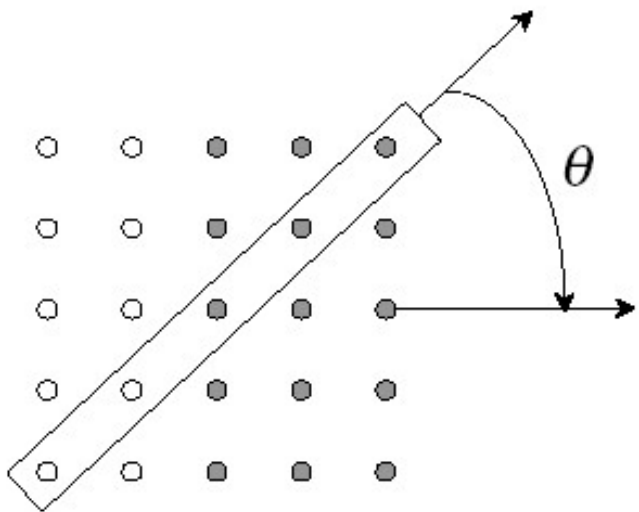
$$\frac{198 + 106 + 90 + 209 + 106 + 99 + 196 + 99 + 115}{9} = 135$$

- Edge has been blurred

Good news: Averaging on only one side of the edge (over 6 pixels) results in a filtered pixel value of 102.5 → Selective averaging

Directional Smoothing

- **Protect edges from blurring:**
 - Calculate averages in several directions
 - Select the direction with the smallest difference between average and noisy pixel value
 - Noisy pixel replaced by average in selected direction



Example:

Vertical = 102.2

Horizontal = 140.8

45 degree = 138.8

135 degree = 144.6

204	194	107	101	96
196	198	106	90	94
196	209	106	99	94
196	196	99	115	101
206	202	93	107	100

K-Nearest Neighbor (K-NN) Noise Removal

- Center rectangular window on pixel P
- Average the K pixels in the window whose values are closest to value of P

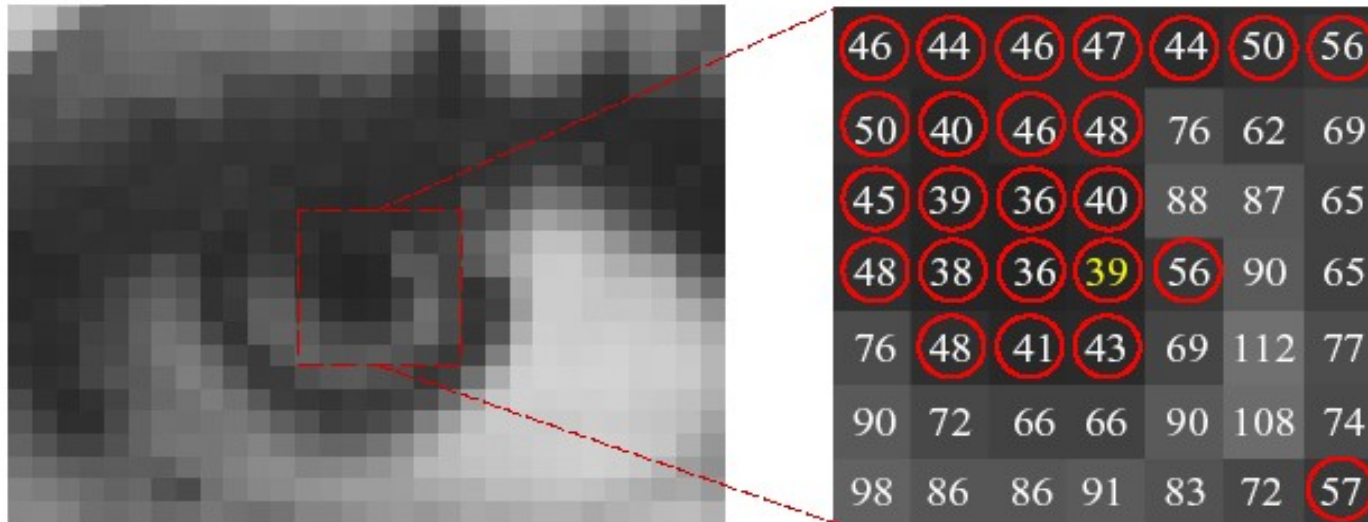
For $K=6$,
the average
is 102.5.

204	194	107	101	96
196	198	106	90	94
196	209	106	99	94
196	196	99	115	101
206	202	93	107	100

$K=6$ { 106
106
99
99
115
90
196
198
209

- User defines window size
- Larger window: more complexity, often better performance

K-NN Noise Removal Example



Window size: $7 \times 7 = 49$ pixels; $K = 24$

Pixel value before noise removal = 39

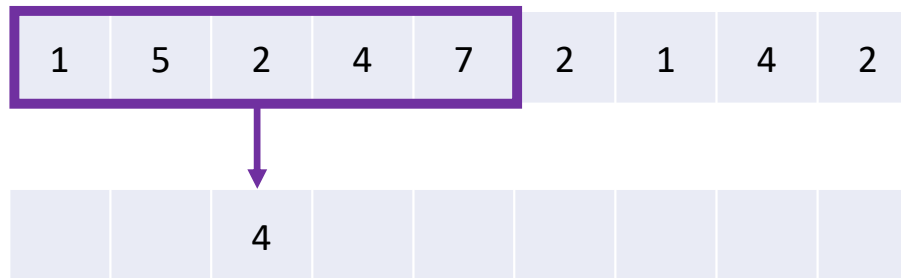
Pixel value after K -NN noise removal = 45

K-NN Noise Removal Example (31x31 window, K=480)



Median Filtering

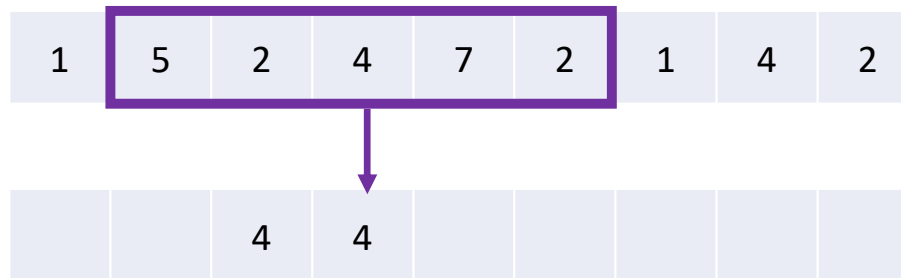
- The *median* of a discrete sequence $a_1, a_2, \dots, a_N$ for N odd is that member of the sequence for which $\frac{N-1}{2}$ elements are smaller or equal in value, and $\frac{N-1}{2}$ elements are larger or equal in value
- 1-D median filtering (MF): For a sliding odd-sized window: In the output sequence, the center pixel in the window is replaced by the median of the pixels in the window



1, 2, 4, 5, 7

Median Filtering

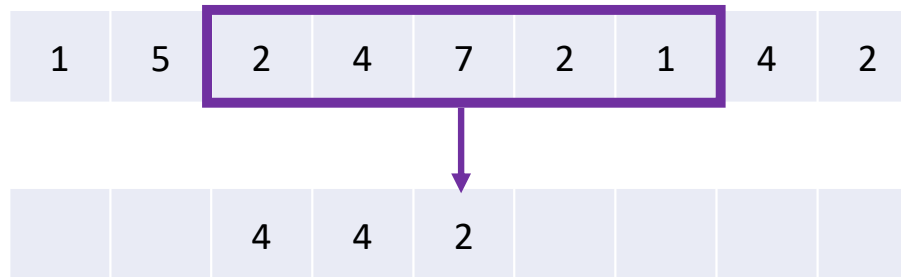
- The *median* of a discrete sequence $a_1, a_2, \dots, a_N$ for N odd is that member of the sequence for which $\frac{N-1}{2}$ elements are smaller or equal in value, and $\frac{N-1}{2}$ elements are larger or equal in value
- 1-D median filtering (MF): For a sliding odd-sized window: In the output sequence, the center pixel in the window is replaced by the median of the pixels in the window



2, 2, 4, 5, 7

Median Filtering

- The *median* of a discrete sequence $a_1, a_2, \dots, a_N$ for N odd is that member of the sequence for which $\frac{N-1}{2}$ elements are smaller or equal in value, and $\frac{N-1}{2}$ elements are larger or equal in value
- 1-D median filtering (MF): For a sliding odd-sized window: In the output sequence, the center pixel in the window is replaced by the median of the pixels in the window



1, 2, 2, 4, 7

Median Filter Properties

Unlike the mean value of a sequence, the median is:

- always a member of the sequence
- insensitive to changes in the tail of the distribution: $med\{1, 2, 3\} = med\{1, 2, 300\}$
- a nonlinear operation, i.e., the median of the sum of two sequences is not necessarily the sum of the medians
- Can analyze only a little:

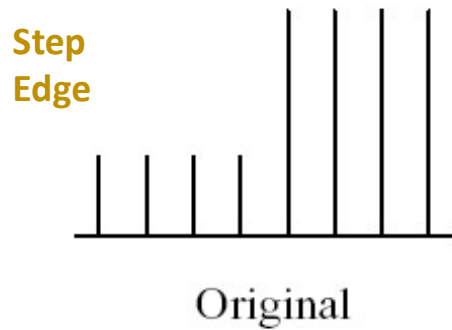
$$med\{K + f(j)\} = K + med\{f(j)\}$$

$$med\{K f(j)\} = K med\{f(j)\}$$

If the window size is even, median is the average of the 2 values in the middle

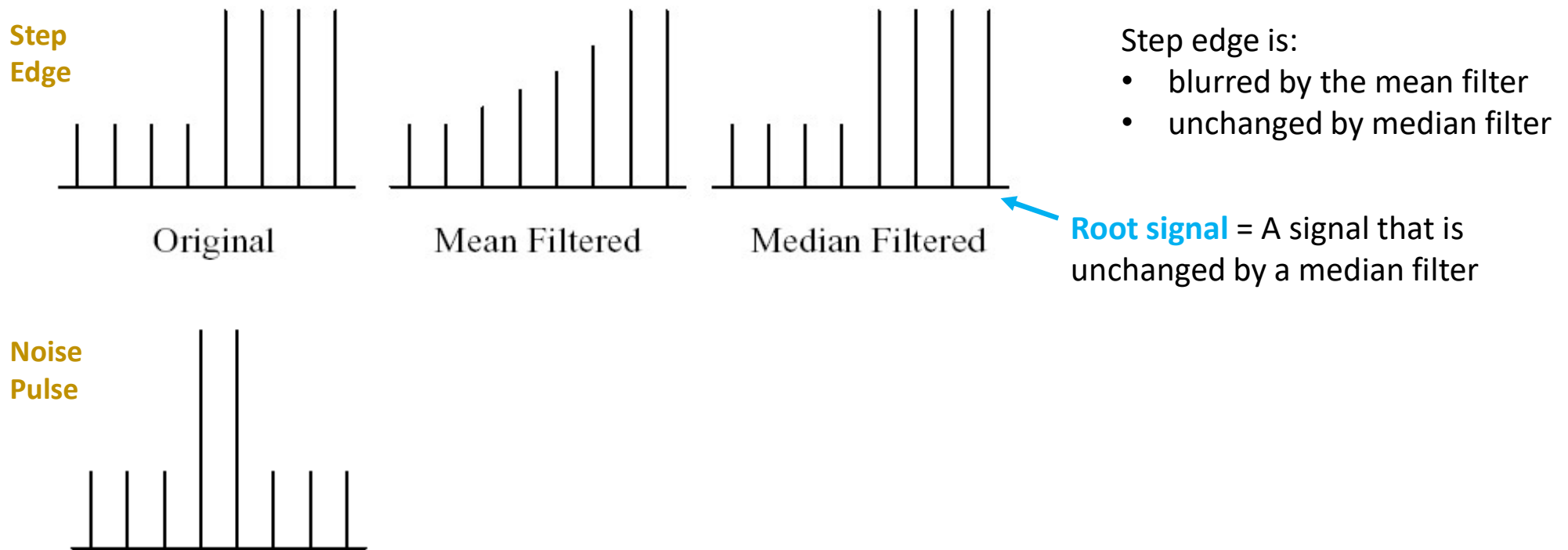
Median Filtering: Effect on Edges and Pulses

Use 1-D 5-point filters (mean filter and median filter) on a 1-D edge and pulse:



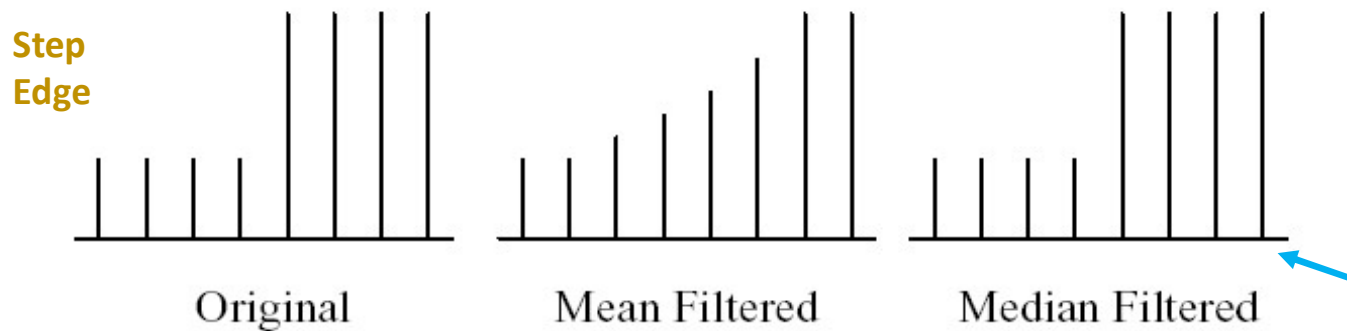
Median Filtering: Effect on Edges and Pulses

Use 1-D 5-point filters (mean filter and median filter) on a 1-D edge and pulse:



Median Filtering: Effect on Edges and Pulses

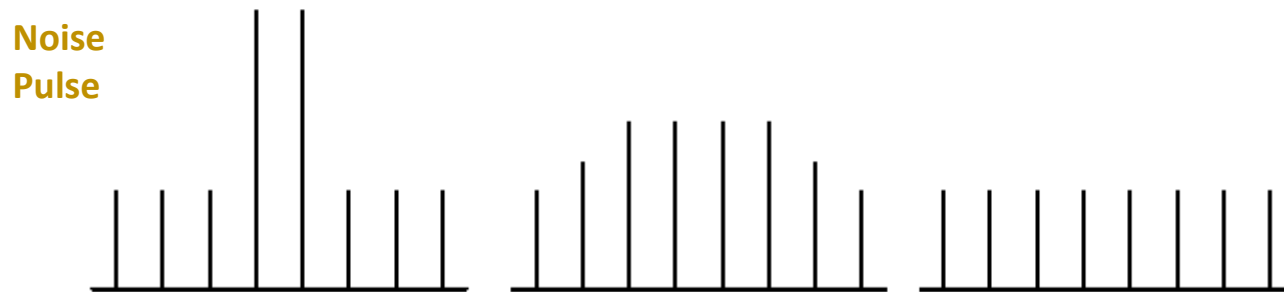
Use 1-D 5-point filters (mean filter and median filter) on a 1-D edge and pulse:



Step edge is:

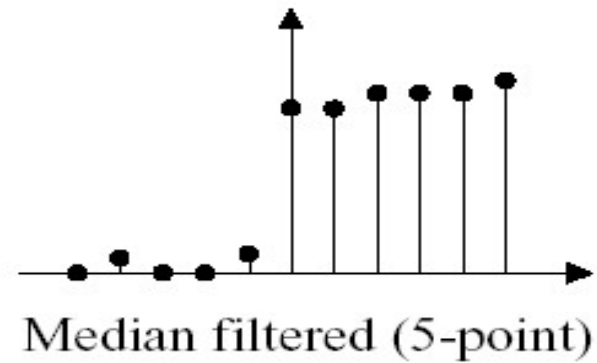
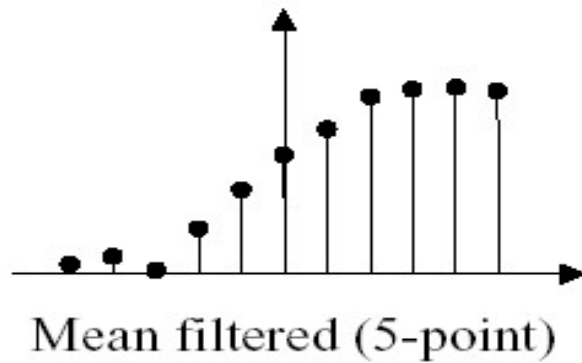
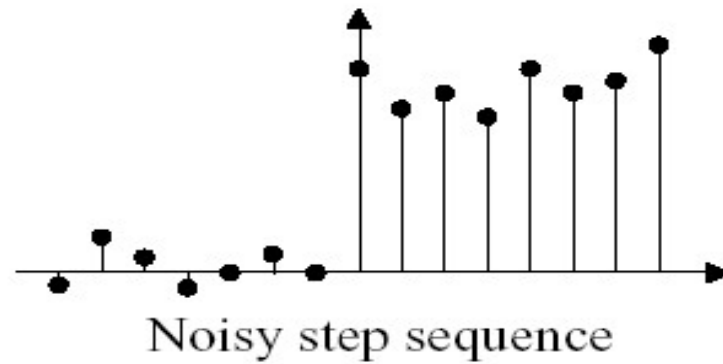
- blurred by the mean filter
- unchanged by median filter

Root signal = A signal that is unchanged by a median filter



For a median filter of size $2n+1$, pulses up to size n pixels are completely suppressed

Median Filtering Example



Median Filtering in 2D

Can extend MF to 2D by using a window of some desired shape, e.g., 3x3

$$\text{Unit 1-D step: } u(n) = \begin{cases} 1, & n \geq 0 \\ 0, & \text{else} \end{cases}$$

$$\text{Unit 2-D step: } u(n, m) = \begin{cases} 1, & n \geq 0, m \geq 0 \\ 0, & \text{else} \end{cases}$$

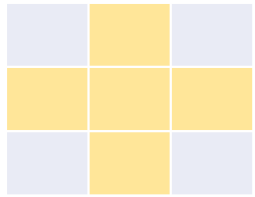
0	0	N	N	N
0	0	N	N	N
0	0	N	N	N
0	0	0	0	0
0	0	0	0	0



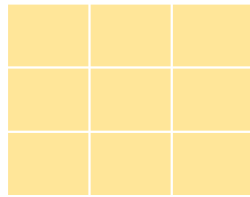
0	0	N	N	N
0	0	N	N	N
0	0	0	N	N
0	0	0	0	0
0	0	0	0	0

- 3x3 MF takes out the corner
- 5x5 MF takes out more
- Unchanged far from origin

Median Filtering: Different Shapes

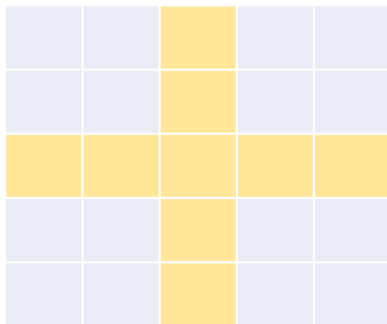


5-pt

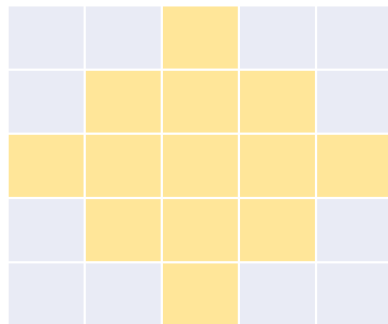


9-pt

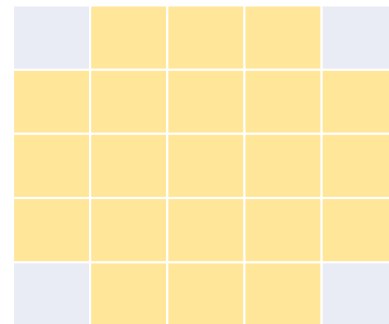
- Can think of these as intermediate sizes of MFs
- But there's a more profound effect...



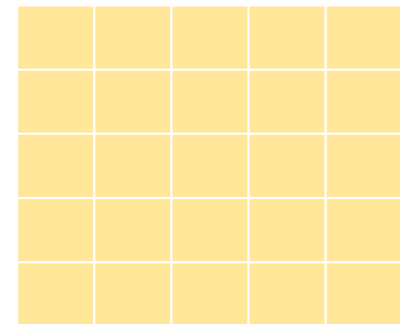
9-pt



13-pt

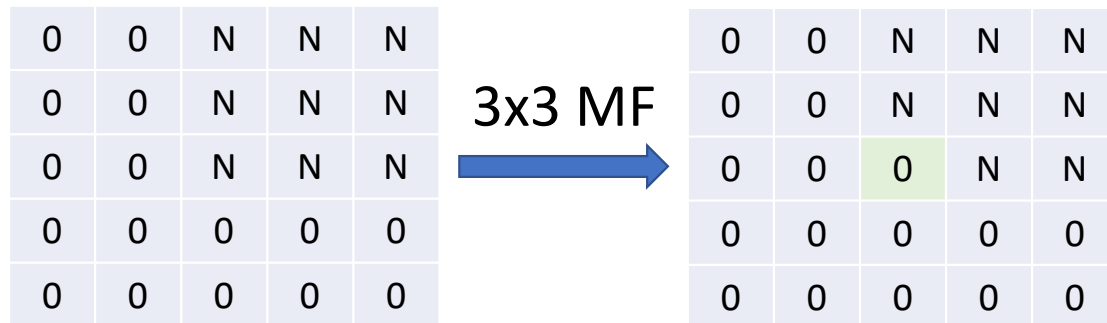


21-pt

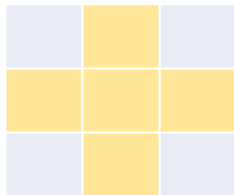


25-pt

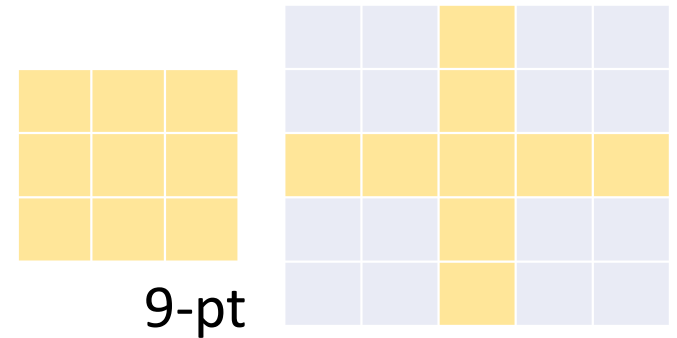
Median Filtering in 2D



What does the plus shape do?



5-pt



9-pt

0	0	N	0	0
0	0	N	0	0
0	0	N	0	0
0	0	N	0	0
0	0	N	0	0

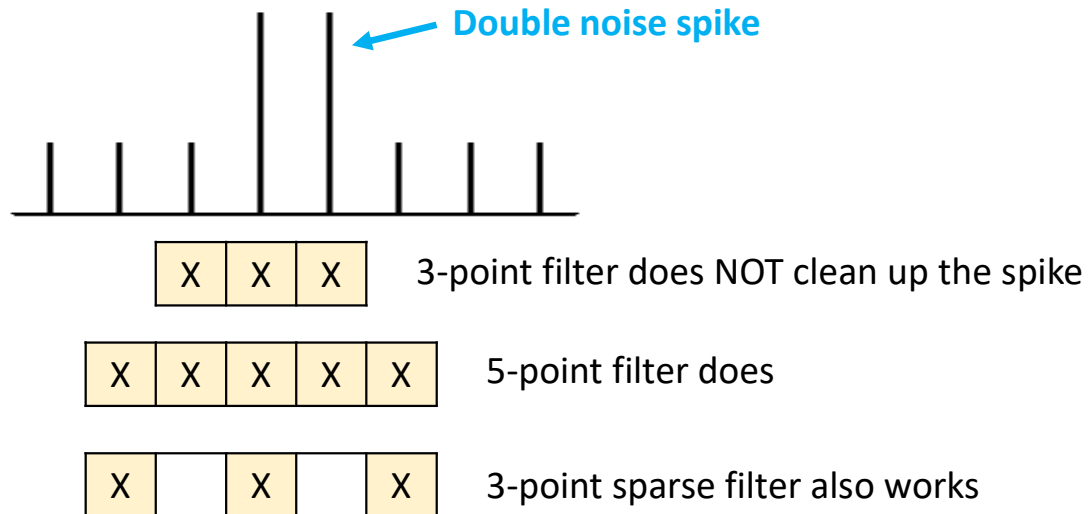
Separable Median Filtering

Can filter 1-D vertically, then horizontally
What does this do to a step?

0	0	N	N	N
0	0	N	N	N
0	0	N	N	N
0	0	0	0	0
0	0	0	0	0

Sparse Median Filtering

The spatial extent of a MF and the number of pixels used in the computation are related but not identical



	X		X	
X				X
		X		
X				X
	X		X	

Sparse 5x5 with only 9 points

What does this do to a 3x3 noise blob?
What does a 3x3 MF do to it?

Alpha-Trimmed Mean Filter

- Mean filter does poorly for outlier noise, well for Gaussian
- Median filter is the other way around
- How can we get some advantages of both?

$$\underbrace{a_1 \leq a_2}_{\times} \leq a_3 \leq \cdots \leq a_{(N+1)/2} \leq \cdots \leq a_{N-2} \leq \underbrace{a_{N-1} \leq a_N}_{\times}$$

- Delete some percentage of the lowest values and highest values from the set
- Take the mean of the remaining values

Other Order Statistic Filters

- Median filter is called an “order statistic” filter since it involves putting the values in neighborhood S in sorted order, and taking the value at the 50th percentile
- Can take different percentiles

Max Filter

100th percentile: Output = $\max_S\{f(x,y)\}$

- Removes pepper noise
- Erodes dark areas adjacent to bright areas
- Brightens image overall

Min Filter

0th percentile: Output = $\min_S\{f(x,y)\}$

- Removes salt noise
- Erodes bright areas adjacent to dark areas
- Darkens image overall

Adaptive Median Filtering: Example Algorithm

$Z_{\min}$ & $Z_{\max}$ = min & max values in S

Z_{med} = median value in S

Z_{xy} = value at (x,y)

$S_{\max}$ = max allowed size of S

Level A: If $Z_{\min} < Z_{\text{med}} < Z_{\max}$ go to Level B  Z_{med} is not an extreme value

Else increase size of S

If $S \leq S_{\max}$ = repeat Level A

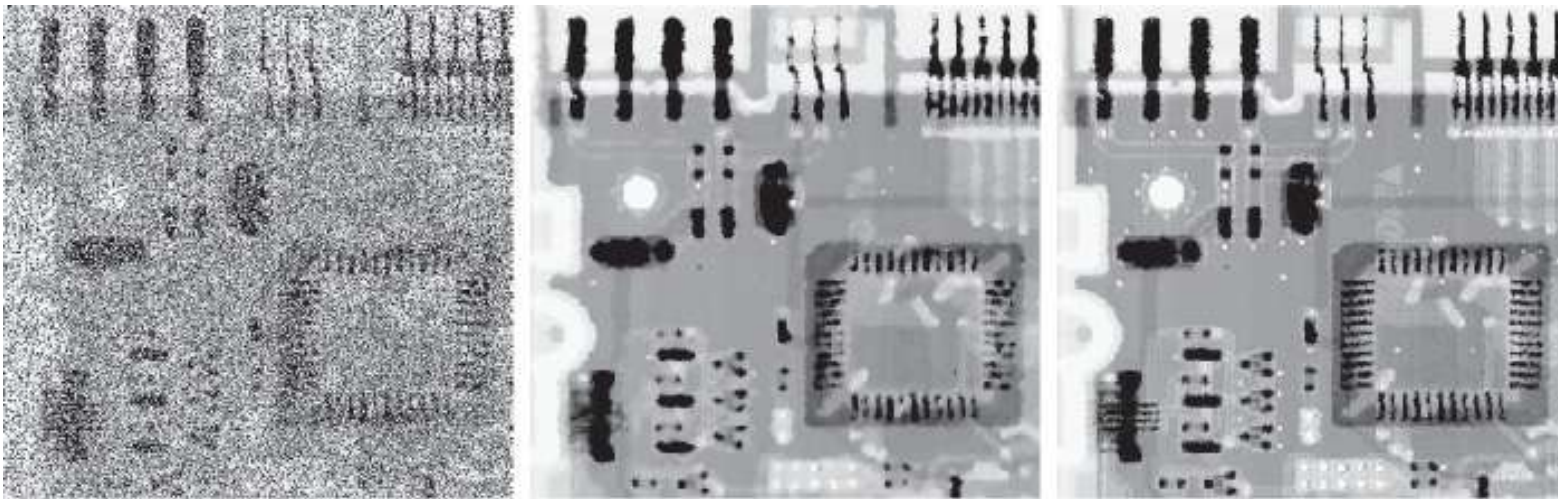
Else, output Z_{med}

Level B: If $Z_{\min} < Z_{xy} < Z_{\max}$ output Z_{xy}  Z_{xy} is not an extreme value

Else output Z_{med}

Adaptive Median Filtering Example

Gonzalez & Woods Fig 5.14



a b c

FIGURE 5.14 (a) Image corrupted by salt-and-pepper noise with probabilities $P_s = P_p = 0.25$. (b) Result of filtering with a 7×7 median filter. (c) Result of adaptive median filtering with $S_{\max} = 7$.

Edge Sharpening

Goal: highlight fine detail, sharpen edges

If lowpass filtering blurs, maybe highpass filtering sharpens?

How to implement a highpass or highboost filter?

LPF $\leftrightarrow$ averaging $\leftrightarrow$ integrating

HPF $\leftrightarrow$ differencing $\leftrightarrow$ differentiating

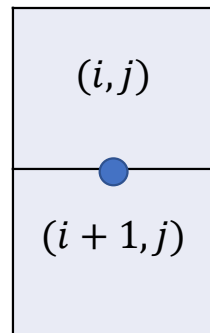
First Differences

First differences in x and y are defined as:

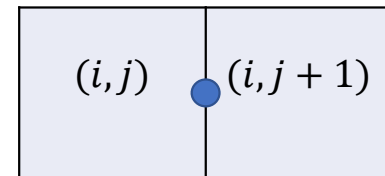
$$\Delta_x f(i, j) = f(i + 1, j) - f(i, j)$$

$$\Delta_y f(i, j) = f(i, j + 1) - f(i, j)$$

$$\Delta_x \sim \frac{\partial f}{\partial x}$$



$$\Delta_y \sim \frac{\partial f}{\partial y}$$



Second differences

Second differences in x and y are defined as:

$$\begin{aligned}\Delta_x^2 f(i, j) &= \Delta_x f(i, j) - \Delta_x f(i - 1, j) \\ &= f(i + 1, j) - f(i, j) - f(i, j) + f(i - 1, j) \\ &= -2f(i, j) + f(i + 1, j) + f(i - 1, j)\end{aligned}$$

$$\begin{aligned}\Delta_y^2 f(i, j) &= \Delta_y f(i, j) - \Delta_y f(i, j - 1) \\ &= f(i, j + 1) - f(i, j) - f(i, j) + f(i, j - 1) \\ &= -2f(i, j) + f(i, j + 1) + f(i, j - 1)\end{aligned}$$

$$\Delta_x^2 \sim \frac{\partial^2 f}{\partial x^2}$$

$(i - 1, j)$	+1
(i, j)	-2
$(i + 1, j)$	+1

+1	-2	+1
----	----	----

Behavior of Δ_x and Δ_x^2

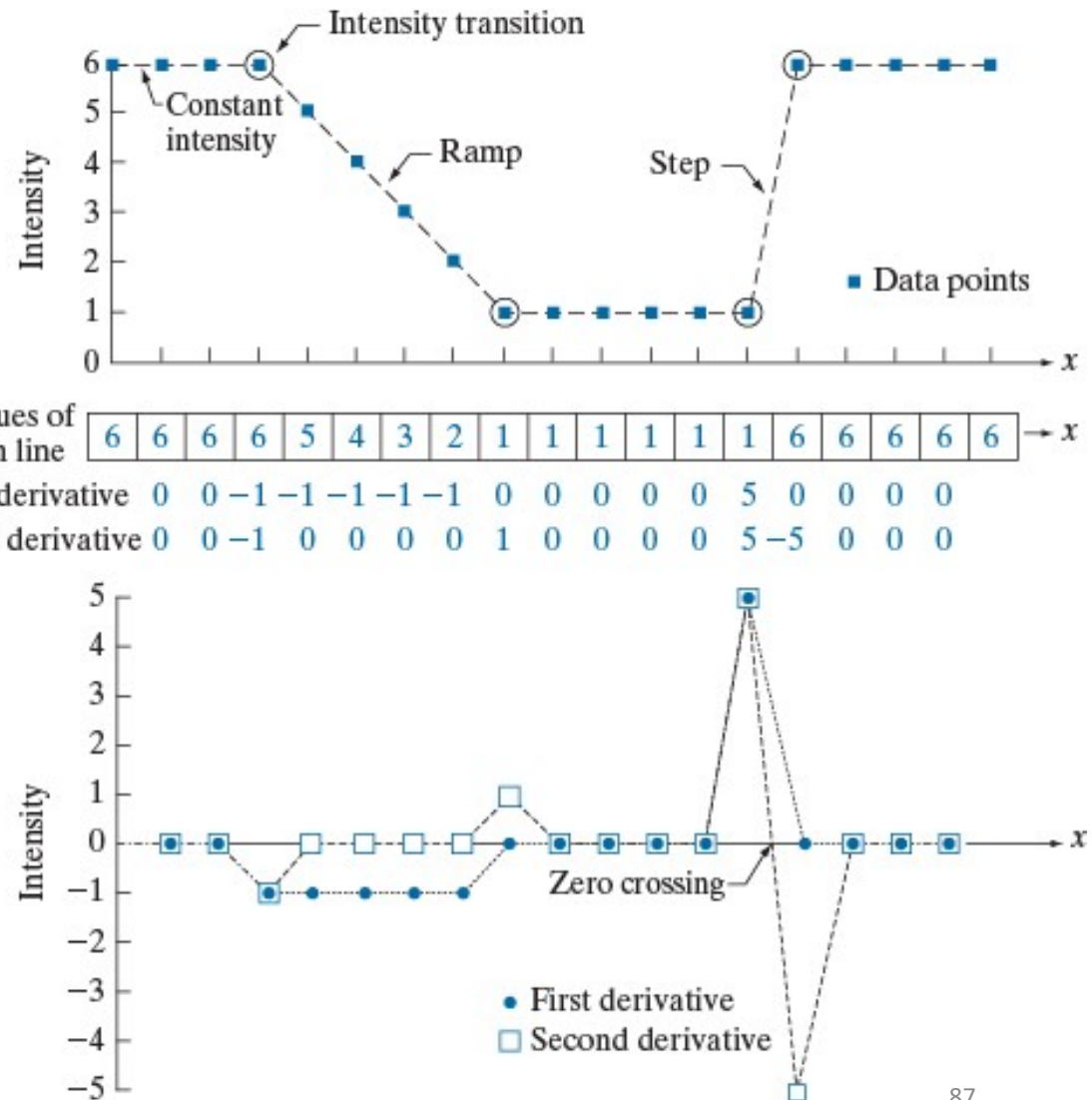
$$\Delta_x f(i, j) = f(i + 1, j) - f(i, j)$$

1. Zero for constant intensity
2. Nonzero at onset of step or ramp
3. Nonzero along ramp

$$\Delta_x^2 f(i, j) = -2f(i, j) + f(i + 1, j) + f(i - 1, j)$$

1. Zero for constant intensity
2. Nonzero at onset/end of step or ramp
3. Zero along ramp

Fig. 3.50 from GW



Laplacian Operator

$$\frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2} \quad \text{Simplest isotropic linear derivative operator}$$

Isotropic means the same in every direction

If (x,y) is an unrotated coordinate system and (x',y') is a rotated coordinate system; they are related by:

$$x = x' \cos \theta - y' \sin \theta$$

$$y = x' \sin \theta + y' \cos \theta$$

$$\frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2} = \frac{\partial^2 f}{\partial x'^2} + \frac{\partial^2 f}{\partial y'^2}$$

Digital Laplacian

We can form a digital version of the Laplacian $\Delta_x^2 + \Delta_y^2 \sim \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}$

$$\Delta_x^2 f(i, j) + \Delta_y^2 f(i, j) = -4f(i, j) + f(i + 1, j) + f(i - 1, j) + f(i, j + 1) + f(i, j - 1)$$

0	1	0
0	-2	0
0	1	0

+

0	0	0
1	-2	-1
0	0	0

=

0	1	0
1	-4	1
0	1	0

Digital Laplacian

$$\nabla^2 f(i, j)$$

Could also define an 8-neighbor version:

1	1	1
1	-8	1
1	1	1

Many forms of the digital Laplacian

We could have started with the definitions going the other way:

$$\Delta_x f(i, j) = f(i, j) - f(i + 1, j)$$

Would come out with:

0	-1	0
-1	4	-1
0	-1	0

0	1	0
1	-4	1
0	1	0

1	1	1
1	-8	1
1	1	1

GW

0	-1	0
-1	4	-1
0	-1	0

-1	-1	-1
-1	8	-1
-1	-1	-1

Normalized versions??

 $\frac{1}{4}$

0	1	0
1	-4	1
0	1	0

 $\frac{1}{8}$

1	1	1
1	-8	1
1	1	1

Digital Laplacian

0	1	0
1	-4	1
0	1	0

Is this behaving as a highpass filter?

100	100	100	100	100
100	100	100	100	100
100	100	100	100	100
100	100	100	100	100
100	100	100	100	100



0	0	0	0	0
0	0	0	0	0
0	0	0	0	0
0	0	0	0	0
0	0	0	0	0

Constant
intensity is
zeroed out

80	90	100	110	120
80	90	100	110	120
80	90	100	110	120
80	90	100	110	120
80	90	100	110	120



0	0	0	0	0
0	0	0	0	0
0	0	0	0	0
0	0	0	0	0
0	0	0	0	0

Steady ramp is
zeroed out

Effect on a High Spatial Frequency

0	1	0
1	-4	1
0	1	0

$\frac{1}{4}$	0	-1	0
	-1	4	-1
	0	-1	0

0	0	0	0	0
200	200	200	200	200
0	0	0	0	0
200	200	200	200	200
0	0	0	0	0



400	400	400	400	400
-400	-400	-400	-400	-400
400	400	400	400	400
-400	-400	-400	-400	-400
400	400	400	400	400



100	100	100	100	100
100	100	100	100	100
100	100	100	100	100
100	100	100	100	100
100	100	100	100	100

+

-100	-100	-100	-100	-100
100	100	100	100	100
-100	-100	-100	-100	-100
100	100	100	100	100
-100	-100	-100	-100	-100

1. Could use the other definition of Laplacian
2. Could use normalized version

-100	-100	-100	-100	-100
100	100	100	100	100
-100	-100	-100	-100	-100
100	100	100	100	100
-100	-100	-100	-100	-100

Laplacian for Edge Sharpening

$$g(i,j) = f(i,j) + c[\nabla^2 f(i,j)]$$

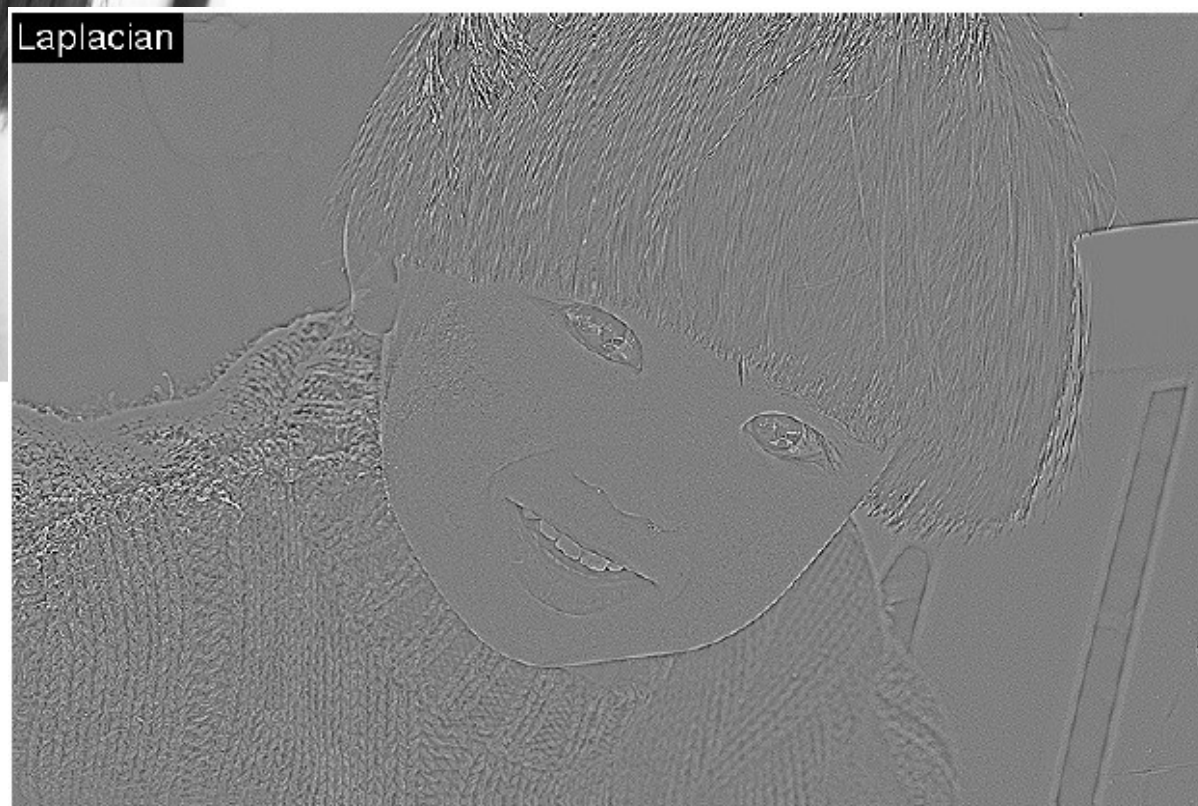
0	1	0
1	-4	1
0	1	0

Take c negative (e.g., -1)

0	-1	0
-1	4	-1
0	-1	0

Take c positive

Sharpened image = Original image + highpass portion



Courtesy of Pam Cosman



$$g(i,j) = f(i,j) + \frac{1}{2} [\nabla^2 f(i,j)]$$

Sharpened = Original + 1/2 Laplacian



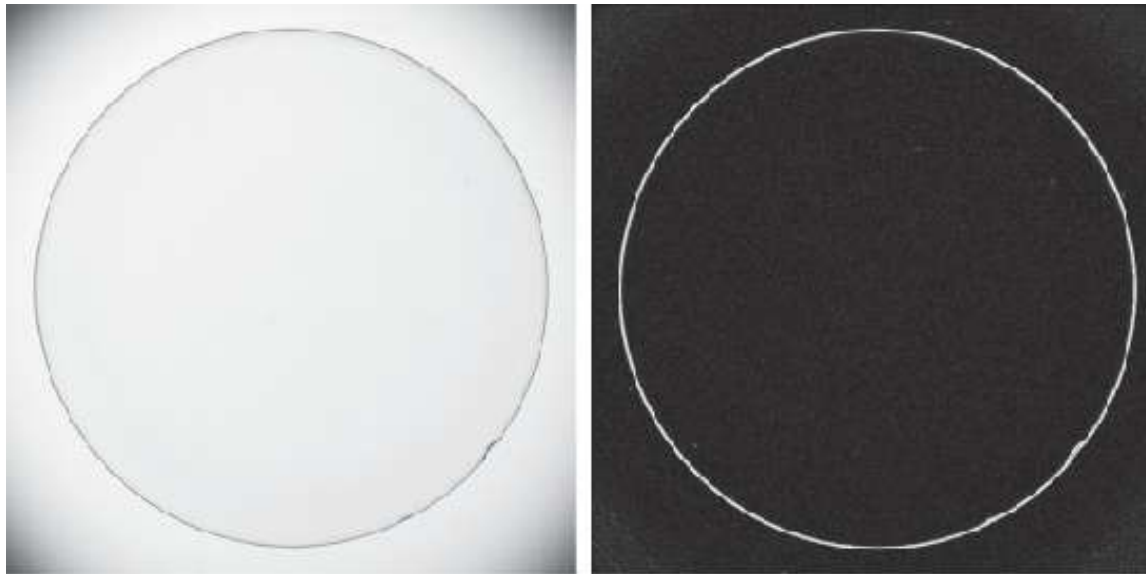


$$g(i,j) = f(i,j) + [\nabla^2 f(i,j)]$$

Sharpened = Original + Laplacian



Magnitude of gradient vector



a b

FIGURE 3.57

(a) Image of a contact lens (note defects on the boundary at 4 and 5 o'clock).
(b) Sobel gradient.
(Original image courtesy of Perceptics Corporation.)

$$\|\nabla f\| = \sqrt{\left(\frac{\partial f}{\partial x}\right)^2 + \left(\frac{\partial f}{\partial y}\right)^2}$$

Sharpening

a b
c d

FIGURE 3.52

(a) Blurred image of the North Pole of the moon.

(b) Laplacian image obtained using the kernel in Fig. 3.51(a).

(c) Image sharpened using Eq. (3-63) with $c = -1$.

(d) Image sharpened using the same procedure, but with the kernel in Fig. 3.51(b).

(Original image courtesy of NASA.)

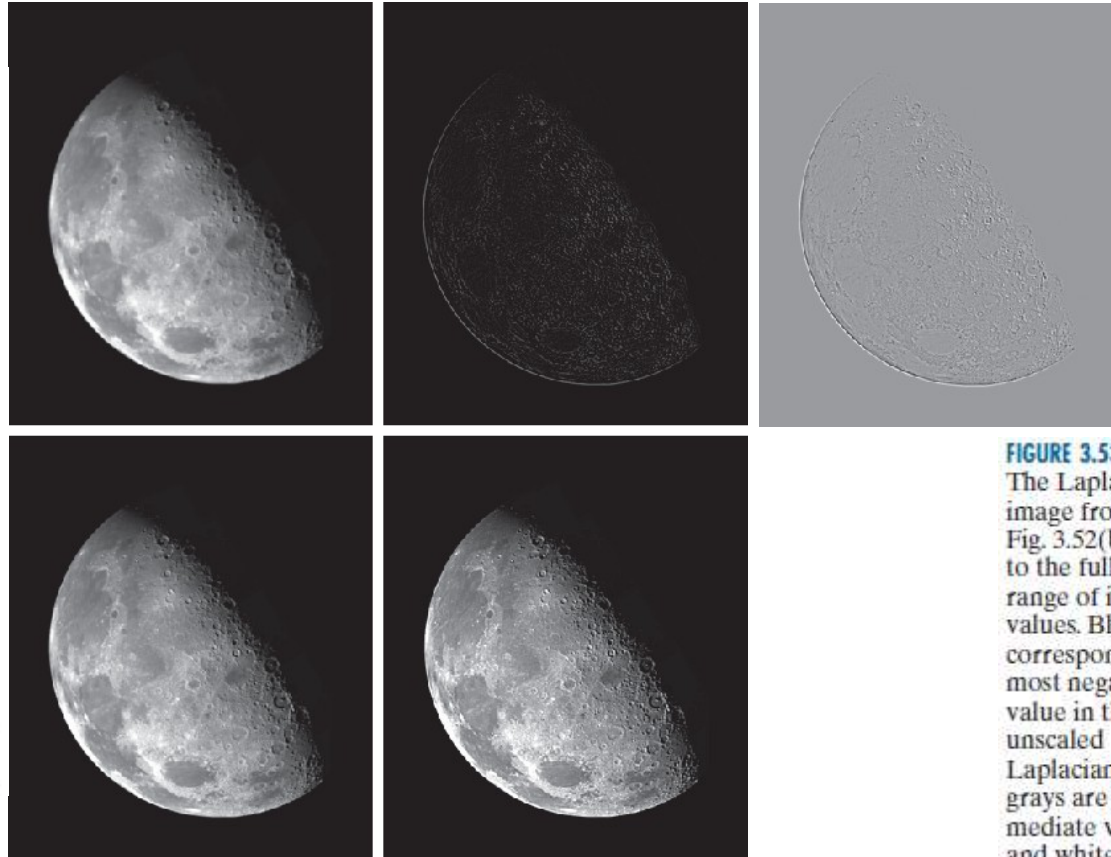
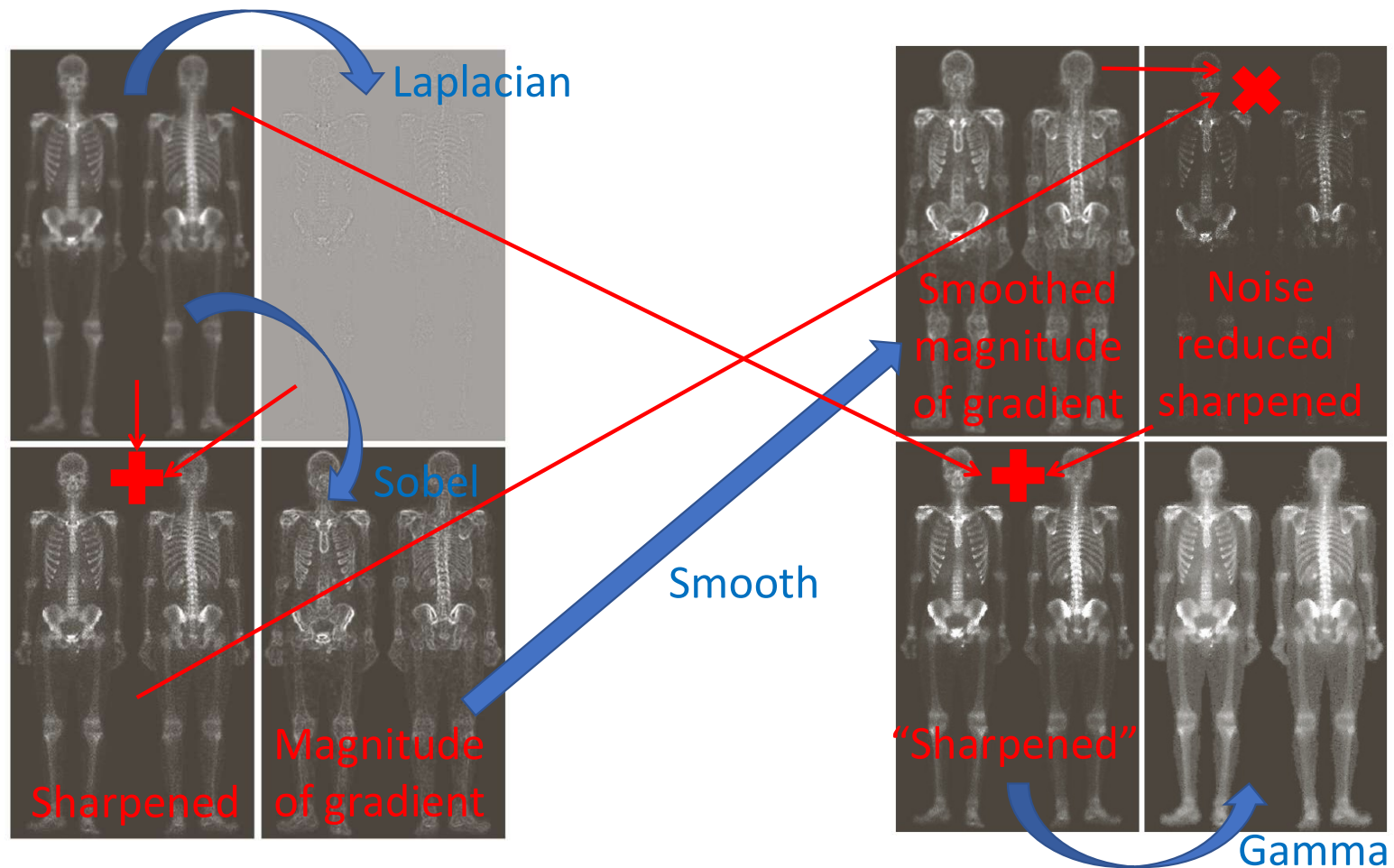


FIGURE 3.53

The Laplacian image from Fig. 3.52(b), scaled to the full $[0, 255]$ range of intensity values. Black pixels correspond to the most negative value in the unscaled Laplacian image, grays are intermediate values, and white pixels corresponds to the highest positive value.

Combining spatial filtering and intensity transformations



Border padding

v	v	v	v	v	v	v	v	v
v	v	v	v	v	v	v	v	v
v	v	1	2	3	4	5	v	v
v	v	6	7	8	9	10	v	v
v	v	11	12	13	14	15	v	v
v	v	16	17	18	19	20	v	v
v	v	v	v	v	v	v	v	v

Zero padding
when $v = 0$

Constant padding

1	1	1	2	3	4	5	5	5
1	1	1	2	3	4	5	5	5
1	1	1	2	3	4	5	5	5
6	6	6	7	8	9	10	10	10
11	11	11	12	13	14	15	15	15
16	16	16	17	18	19	20	20	20
16	16	16	17	18	19	20	20	20

Replicate padding

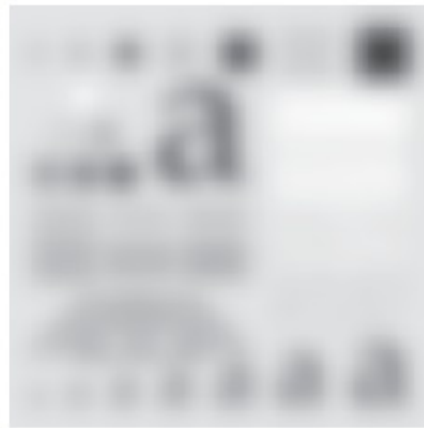
13	12	11	12	13	14	15	14	13
8	7	6	7	8	9	10	9	8
3	2	1	2	3	4	5	4	3
8	7	6	7	8	9	10	9	8
13	12	11	12	13	14	15	14	13
18	17	16	17	18	19	20	19	18
13	12	11	12	13	14	15	14	13

Mirror padding

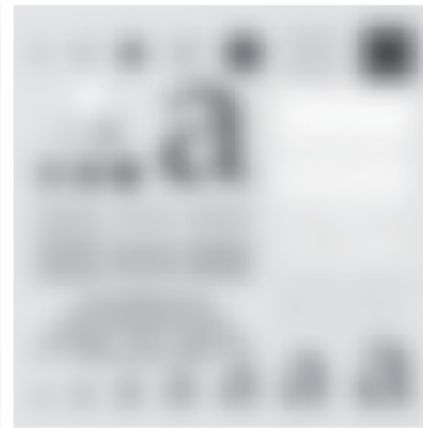
Border padding



Zero
padding



Mirror
padding



Replicate
padding